Study Buddy STEM AI: An AI-Powered Personalized, Adaptive and Multimodal STEM Learning Ecosystem for Sri Lankan Students

INDIVIDUAL THESIS

Elwitigala S.A.I

IT22087638

Individual component: RAG-Based Personalized STEM Storytelling

Group ID: R26-IT-009

BSc (Hons) in Information Technology

Specializing in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

July 2026

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy. I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Elwitigala S.A.I

Student ID:IT22087638

Signature: _____ Date: _____

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Role and name	Signature	Date
Supervisor: : Mr. Aruna Ishara Gamage		
Co-supervisor: Mr. Jeewaka Perera		

ABSTRACT

[Guidance - delete before submission: Maximum 300 words. State the individual problem and gap, aim, research and development methods, principal quantitative or qualitative results, conclusion, and individual contribution. The abstract must stand alone.]

[Write the abstract here as one coherent paragraph or a small number of connected paragraphs.]

Keywords: [three to five keywords, separated by commas]

ACKNOWLEDGEMENTS

[Guidance - delete before submission: Acknowledge supervision and support that directly assisted your individual research and development work. State any funding or substantial technical assistance.]

[Replace this paragraph with the required content.]

TABLE OF CONTENTS

1 INTRODUCTION 1
1.1 Overall project context 1
1.2 Individual component context 2
1.3 Problem statement 3
1.3.1 Research Problem A: Ungrounded story generation 3
1.3.2 Research Problem B: Personalisation that does not scale 4
1.4 Research gap 4
1.5 Aim and objectives 5
1.5.1 Aim 5
1.5.2 Objectives 5
1.6 Research questions or hypotheses 6
1.7 Scope limitations and assumptions 6
1.7.1 Scope 6
1.7.2 Limitations 6
1.7.3 Assumptions 7
1.8 Individual contribution 7
1.9 Thesis structure 8
2 LITERATURE REVIEW AND RELATED WORK 9
2.1 Review method and scope 9
2.1.1 Search strategy 9
2.1.2 Inclusion and exclusion criteria 10
2.1.3 Categorising the corpus 11
2.2 Domain and theoretical foundations 15
2.2.1 Narrative learning in STEM 15

2.2.2 Student profiling and machine learning classification 15
2.2.3 Information retrieval and retrieval-augmented generation 16
2.2.4 Lesson structure: separating the explanation from the story 16
2.3 Related research 17
2.3.1 Empirical studies of narrative learning in STEM 17
2.3.2 Machine learning for student profiling and classification 18
2.3.3 Story generation with large language models 18
2.3.4 Retrieval-augmented generation for curriculum grounding 19
2.4 Existing systems products and techniques 19
2.4.1 Existing systems and products 19
2.4.2 Techniques used in this research 21
2.5 Gap synthesis and conceptual framework 22
3 RESEARCH METHODOLOGY 24
3.1 Research design 24
3.2 Data participants or study materials 25
3.3 Experimental or study procedure 26
3.4 Measures and evaluation metrics 28
3.5 Data analysis 30
3.6 Ethics privacy security and responsible AI 30
3.7 Reproducibility and research artefacts 31
4 PRODUCT AND SOLUTION DESIGN 33
4.1 Requirements 33
4.2 Component architecture within the complete solution 34
4.3 Design alternatives and decisions 39
4.4 Detailed design 42
4.5 Standards and professional practice 47

5 IMPLEMENTATION INTEGRATION AND TESTING 49
5.1 Development environment and technology stack 49
5.2 Component implementation 50
5.3 Integration with the other components 52
5.4 Verification and software testing 53
5.5 Deployment and operation 54
6 RESULTS AND VALIDATION 56
6.1 Research results 56
6.2 Product and technical results 59
6.3 User or stakeholder validation 60
6.4 Integration results 61
6.5 Summary of findings 61
7 DISCUSSION 63
7.1 Interpretation of findings 63
7.2 Comparison with literature baselines and existing solutions 64
7.3 Research contribution 64
7.4 Product contribution 65
7.5 Limitations and threats to validity 65
7.6 Implications 66
8 CONCLUSION AND FUTURE DIRECTIONS 67
8.1 Conclusion 67
8.2 Achievement of objectives 68
8.3 Individual research and product contributions 69
8.4 Future directions 69
REFERENCES 70
GLOSSARY 73

APPENDICES 74
Appendix A Research instruments or extended results 74
Appendix B Academic integrity and AI use disclosure 74

LIST OF FIGURES

Figure 1:Overview of the Narrative Learning component 3
Figure 2:Architecture of the complete solution, with the Narrative Learning component highlighted 35
Figure 3:Internal architecture of the Narrative Learning component 37
Figure 4:Sequence of events in one session 46

LIST OF TABLES

Table 1:Literature corpus by review theme 11
Table 2:Comparison of existing systems with this research 20
Table 3:Established techniques used in this research 21
Table 4:Research gaps and the response of this research 23
Table 5:How the component is evaluated 24
Table 6:Training records for each story theme 26
Table 7:Measures used in the evaluation 28
Table 8:Research artefacts 31
Table 9:Functional requirements 33
Table 10:Non-functional requirements 34
Table 11:Requests handled by the component 38
Table 12:Benchmark comparison of the two local models 40
Table 13:Summary of design decisions 41
Table 14:Survey answer fields 42
Table 15:Lesson output fields 43
Table 16:Classifier settings 43
Table 17:Rules set by the prompt 45
Table 18:Language model settings 45
Table 19:Standards and practices applied 47
Table 20:Software and library versions 49
Table 21:Models used by the component 50
Table 22:Configuration settings 50
Table 23:Location of the component's code and files 51
Table 24:Non-functional checks 53
Table 25:Requirements covered by each kind of test 54
Table 26:Accuracy of the theme classifier 56
Table 27:Confusion matrix of the theme classifier 56
Table 28:Results of the lesson generation benchmark 57
Table 29:Llama 3.1 results for each benchmark case 58
Table 30:Results of the non-functional tests 59

Table 31:Non-functional requirements and their results..... 60
Table 32:Summary of findings..... 61
Table 33:Achievement of objectives..... 68

LIST OF ABBREVIATIONS

[Guidance - delete before submission: List abbreviations alphabetically. Define each abbreviation at first use in the main text even when it appears here.]

Abbreviation	Meaning
[ABC]	[Full term]
[ABC]	[Full term]
[ABC]	[Full term]

1 INTRODUCTION

1.1 Overall project context

In Sri Lanka, students preparing for the G.C.E. Ordinary Level examination meet STEM subjects mostly through the official textbooks and past papers. The content is dense, and a textbook explains each topic one way, but students do not all learn the same way.

A general AI chatbot does not solve this. It often invents facts, brings in content from outside the syllabus, or uses wording that differs from the textbook. For a student preparing for an examination, that is a risk.

Our group project, "An AI-Assisted STEM Learning Ecosystem for Sri Lankan G.C.E. O/L Students", takes a different approach. The system looks up the official textbook first, and the model is prompted to answer only from the passages it finds. This approach is called retrieval-augmented generation (RAG). The system is designed for STEM learning. For this research we chose Science as the subject and used the official Grade 10–11 pupil books and teacher guides.

The system is one web application with four modules, each for a different way of learning. Voice Tutor answers typed or spoken questions from the syllabus. Narrative Learning matches a student to a story theme and teaches the topic through a story. Adaptive Quiz adjusts to how the student behaves during a quiz. Knowledge Maps check a student's notes against the syllabus and builds mind maps from them. Narrative Learning is my individual component.

1.2 Individual component context

My individual component is Narrative Learning. It teaches Grade 10 and 11 Science through short stories that match each student's interests, while keeping every fact tied to the official textbook.

A textbook presents a scientific law in the abstract, with little connection to a student's own life. Research shows that presenting science through stories can improve recall, engagement and understanding [1], [2]. However, a teacher cannot write a personal story for every student in a class.

Narrative Learning automates this in two stages. It combines a machine learning classifier with retrieval-augmented generation [3].

In the first stage, the student completes a short survey about their grade, free-time interest, career aspiration, how much they struggle with science, and the kind of story they prefer. A Random Forest classifier then uses the interest and aspiration to place the student in one of five story themes: Action & Competition, Creative Expression, Sci-Fi & Innovation, Nature & Discovery, or Logic & Mystery. Similar machine learning methods have been used to personalise learning [4].

In the second stage, the student selects a topic, and the component retrieves the matching passages from the textbook. The language model is told to write only from those passages. Grounding the model in this way reduces the risk of invented facts [5].

The lesson has two parts. **TELL** is a plain classroom explanation of the law, why it works, and its formulae, with no characters or plot. **SHOW** is a three-paragraph story set in everyday Sri Lanka. It opens with a hook tied to the student's interest, moves to a scene where science happens, and ends with a character stating the textbook law.

The component runs on Ollama with the Llama 3.1 model on the local machine. Student text stays on that machine, and there is no per-token cost.

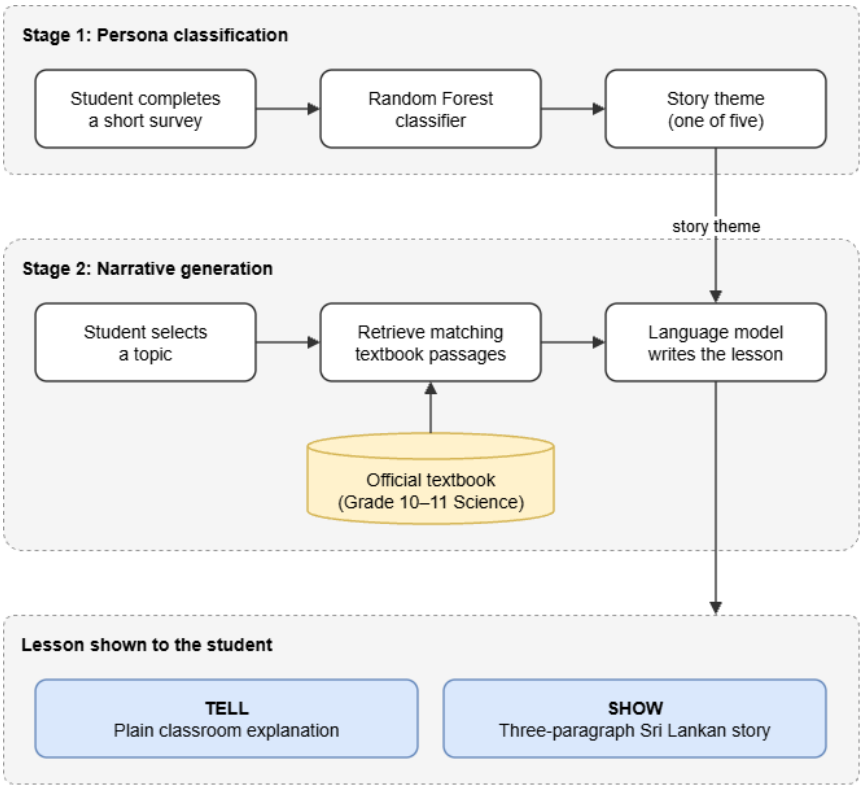


Figure 1:Overview of the Narrative Learning component

1.3 Problem statement

1.3.1 Research Problem A: Ungrounded story generation

A language model that writes without a source to check against can produce confident text that is wrong [5], [6]. When the model is asked to turn science into a story, it often invents laws, numbers or examples that are not in the Grade 10–11 textbook. It may also leave out the definition the student needs.

This matters in an exam-oriented subject such as G.C.E. O/L Science. If a story misstates a law or uses different wording from the textbook, the student may learn the wrong material. A personalised story with no textbook behind it is entertainment, not syllabus teaching.

Retrieval-augmented generation has been studied as a way to reduce these errors in education [7], [8]. The problem this research addresses is how to apply it to story writing, so that a story stays engaging while every fact in it comes from the official textbook.

1.3.2 Research Problem B: Personalisation that does not scale

The simple ways to personalise a story are to write it by hand or to match it to a single keyword. A student who likes cricket gets a cricket story. This is too crude, because a hobby is not the same as a taste in stories. A cricket fan may still prefer a story that opens like a mystery.

Writing by hand does not scale either. A teacher cannot write a different story for every student in a class, each with their own interests and career aspirations.

Machine learning has already been used to identify students' learning styles and to adapt content to them[4], [9]. Recent work also uses language models to personalise learning content and to build interactive stories [3], [10]. What is missing is an automatic way to place a student in a story theme from a short survey, and then use that theme to shape a lesson with a fixed teaching structure.

1.4 Research gap

Earlier studies show that stories help students learn science[1], [2]. Other studies have tested retrieval-augmented generation for question answering on school textbooks [7], [11]. Little work combines the two. I identified three gaps.

The first gap is structure. Educational RAG systems typically answer in a single block of text. When that text is a story, the scientific law can get lost inside the plot. To my knowledge, no pipeline keeps the two apart, with a plain explanation of the law (TELL) followed by a separate story (SHOW).

The second gap is how the story is personalised. Story tools mostly use fixed templates, or a prompt written without any model of the student. Tang and Hare used a Random Forest to personalise learning in a narrative game. However, few systems use a classifier trained on a student's interest and career aspiration to choose a story theme, and then use that theme to guide a story written from retrieved textbook passages.

The third gap is deployment. Many personalised learning systems depend on paid cloud models, which adds cost and sends student text outside the school. There is little

published work on textbook-restricted story generation that runs entirely on a local machine with freely available models.

1.5 Aim and objectives

1.5.1 Aim

The aim of this research is to design, build and evaluate a Narrative Learning component that teaches Grade 10 and 11 Science through stories matched to each student's interests. Every fact in a story must come from the official textbook, and the component must run on a local machine without paid cloud models.

1.5.2 Objectives

1. To train and evaluate a Random Forest classifier that uses a student's interest and career aspiration to place them in one of five story themes.
2. To build a retrieval engine that fetches matching passages from the official Grade 10–11 Science textbooks, using the component's own index with the shared project index as a backup.
3. To build a generation pipeline that writes each lesson in two separate parts: a plain classroom explanation (TELL) and a three-paragraph story matches the student's theme and interest (SHOW).
4. To integrate the component into the group's web application and run it on a local language model, so that it has no per-token cost and student text stays on the machine.
5. To evaluate the classifier with stratified cross-validation and a confusion matrix, and to evaluate the stories for faithfulness to the textbook, syllabus coverage and match to the student's interest.

1.6 Research questions or hypotheses

This research answers three questions.

- RQ1: How accurately can a Random Forest classifier place a student in a story theme from their interest and career aspiration?
- RQ2: To what extent does retrieving from the official textbook keep generated stories faithful to the syllabus, compared with the same language model with no retrieval?
- RQ3: Do students who learn a topic through a matched story understand it better, and find it more interesting, than students who read the same topic in the textbook?

1.7 Scope limitations and assumptions

1.7.1 Scope

The component is built for Sri Lankan students in Grades 10 and 11 who are preparing for the G.C.E. Ordinary Level examination. The wider system is designed for STEM learning. Because this research had to be completed within one year, the scope is limited to one subject, Science.

The component works in English only. It uses the official English-medium Grade 10–11 Science pupil books and teacher guides as its only source of facts. It produces written lessons, and does not produce audio or illustrations. It is a study aid that supports the teacher and does not replace the teacher.

1.7.2 Limitations

The real survey sample is small, at about 30 students. To train the classifier I mixed these responses with synthetic student records and gave the real responses three times the weight. The classifier results should be read with this in mind.

The classifier uses only the student's interest and career aspiration. The survey also records how much the student struggles with science, but this is used only to set the depth of the explanation, not to choose the theme.

The component runs a local language model that is smaller than the largest commercial models. Its output sometimes has to be regenerated, or replaced with a fallback lesson built from the textbook text. It also needs a computer able to run the model.

1.7.3 Assumptions

This research assumes that students answer the survey honestly and that their answers reflect their real interests. It assumes that the official textbooks are correct, so a story is treated as accurate when it agrees with them. It also assumes that students can read English at the level of the textbook.

1.8 Individual contribution

I designed and built the Narrative Learning component. My contributions to the group project are:

1. I designed the student survey, collected the responses and prepared the training data.
2. I built the persona classifier. This covers cleaning and encoding the survey answers, training the Random Forest model, and evaluating it.
3. I built the retrieval engine. It searches the component's own textbook index, filters out unusable passages such as page headers, and falls back to the shared project index when too little is found.
4. I built the generation pipeline. This covers the prompt that sets the two-part lesson format, the code that reads the model's output, and the retry and fallback steps for when the output is not usable.
5. I built the API that connects the survey, the classifier, retrieval and generation, and integrated it into the group's web application.

1.9 Thesis structure

The rest of this thesis has seven chapters.

Chapter 2 reviews the literature on narrative learning in science, student classification, language models in education, retrieval-augmented generation and locally run language models. It ends by bringing the research gaps together. Chapter 3 describes the research methodology: the study design, the data and participants, the procedure, the evaluation measures and the ethics of the study. Chapter 4 presents the design of the component, including its requirements, its place in the complete system and the design decisions I made.

Chapter 5 covers implementation and integration with the other components, testing and deployment. Chapter 6 reports the results for the classifier, the generated stories, the student study and the integration. Chapter 7 discusses what the results mean, compares them with literature, and sets out the contributions and limitations. Chapter 8 concludes the thesis, reviews how far each objective was achieved, and suggests future directions such as Sinhala stories. The appendices contain the research instruments, extended results and the AI use disclosure.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review method and scope

I carried out a structured literature review to give the Narrative Learning component a basis in existing research. The review covers five areas: teaching STEM through stories, machine learning for classifying students, personalised content generation with language models, retrieval-augmented generation (RAG), and running open language models locally. Its purpose is to find the limits of existing educational systems, to justify the teaching design, and to bring together the research gaps that this component addresses.

2.1.1 Search strategy

I searched for literature with the Elicit research assistant and with direct queries in IEEE Xplore, the ACM Digital Library, SpringerLink, ScienceDirect and Google Scholar. I grouped the search terms under five themes that match the technical needs and research questions of this project.

1. Narrative teaching in STEM education. These searches looked at how stories are used in secondary science learning. Keywords included “narrative learning science education”, “digital storytelling secondary STEM”, “historical discovery narratives science retention” and “concept retention narrative fiction science”.

2. Student profiling and machine learning classification. These searches looked at supervised learning models that predict student preferences. Keywords included “machine learning student learning styles”, “Random Forest classification student persona”, “predicting student profiles adaptive education” and “learning style classification decision trees”.

3. Personalised content generation with language models. These searches looked at adaptive storytelling with generative AI. Keywords included “personalized story generation LLMs”, “adaptive educational narrative generation”, “interest-based learning LLM tutors” and “LLM personalization student engagement”.

4. Retrieval-augmented generation and reducing hallucination. These searches looked at how answers are grounded in curriculum materials. Keywords included “curriculum-aligned RAG textbooks”, “RAG vs GraphRAG math STEM”, “mitigating LLM hallucinations in education” and “groundedness vs preference educational QA”.
5. Local open-source AI deployment and pipeline design. These searches looked at low-cost AI systems that keep data private. Keywords included “local LLM deployment education”, “source-grounded learning object generation”, “privacy-preserving AI tutoring” and “open-weight LLM educational pipeline”.

2.1.2 Inclusion and exclusion criteria

I applied the following criteria when screening the papers, so that the literature would inform the design of a narrative learning engine for Grade 10–11 Science.

A paper was included if it was one of these:

- A peer-reviewed journal article, conference paper or preprint published between 2010 and 2026.
- An empirical study of storytelling, narrative framing or digital fiction in primary, secondary or undergraduate STEM education.
- A study of supervised machine learning algorithms, such as Random Forest, Decision Trees, Naïve Bayes and ensemble methods, for classifying students’ learning styles, hobbies or behaviour.
- A technical study of RAG, prompt constraints, vector retrieval strategies or hallucination reduction in educational language models.
- A study of source-grounded generation of learning items, or of local open-source AI deployment.

A paper was excluded if it was one of these:

- A publication not in English, or one without accessible full text.
- A commercial software advertisement, opinion article or blog post with no peer review or technical validation.
- A study of recreational storytelling or creative text generation with no curriculum or learning constraint.
- A study only of postgraduate or professional training outside STEM, with nothing that transfers to secondary school science.

2.1.3 Categorising the corpus

The final corpus has 24 studies across the five themes. Table 2.1 lists each study with its authors, year, title, theme and main focus.

Table 1:Literature corpus by review theme

Theme	Author(s) and year	Paper title	Main focus
1. Narrative teaching in STEM	Arya & Maul [1]	The role of the scientific discovery narrative in middle school science education: An experimental study	Experiment on recall and understanding in middle school students who read discovery narratives.
	Csikar & Stefaniak [12]	The Utility of Storytelling Strategies in the Biology Classroom	Storytelling strategies for teaching biology.
	Wadasen & Lubrica [13]	Effect of Teacher-Made Narrative Fiction (TMNF) about Covalent Bonding on Students' Achievement, Retention and Interest	Teacher-made narrative fiction on covalent bonding and its effect on achievement, retention and interest.

	Maharaj-Sharma [14]	Using storytelling to teach a topic in physics	Storytelling for teaching a physics topic.
	Tobler et al. [15]	Teaching biology with narratives: examining the impact of affective and cognitive variables on undergraduate student learning	Affective and cognitive factors in undergraduate biology learning with narratives.
	Agyei [16]	Digital storytelling and students' motivation towards learning genetics: A quasi-experimental study	Quasi-experimental study of digital storytelling and motivation in genetics.
2. Student profiling and ML classification	Tang & Hare [17]	A Random Forest Model for Personalized Learning in a Narrative Game	Random Forest model for personalised learning in a narrative game.
	Shrestha & Pokharel [18]	Determining Learning Style preferences of learners	Determining learners' learning style preferences.
	Ikawati et al. [19]	Student Behavior Analysis to Predict Learning Styles Based Felder Silverman Model Using Ensemble Tree Method	Ensemble tree methods for predicting Felder–Silverman learning styles from student behaviour.
	Almahri et al. [20]	Applications of Machine Learning in Education	Review of machine learning applications in education.
	Garonga & Tanduk [21]	Comparison of Naive Bayes, Decision Tree, and Random Forest Algorithms in Classifying Learning Styles of Universitas Kristen Indonesia Toraja Students	Comparison of Naïve Bayes, Decision Tree and Random Forest for classifying learning styles.

	Fernandes et al. [9]	Advancing Personalized and Adaptive Learning Experience in Education with Artificial Intelligence	AI for personalised and adaptive learning.
	Wanniarachchi & Premadasa [4]	Identifying the Learning Style of Students Using Machine Learning Techniques: An Approach of Felder Silverman Learning Style Model (FSLSM)	Machine learning classification of learning styles under the Felder–Silverman model.
3. LLM personalisation and story generation	Yunusov et al. [22]	MirrorStories: Reflecting Diversity through Personalized Narrative Generation with Large Language Models	Personalised narrative generation with language models that reflects reader diversity.
	Lim Jia Yin et al. [3]	Learning in Context: Personalizing Educational Content with Large Language Models to Enhance Student Learning	Personalising educational content with language models to improve learning.
	Ruskov [23]	Our Coding Adventure: Using LLMs to Personalise the Narrative of a Tangible Programming Robot for Preschoolers	Language-model-personalised narrative for a programming robot for preschoolers.
	Cheng et al. [10]	Oak Story: Improving Learner Outcomes with LLM-Mediated Interactive Narratives	Interactive narratives mediated by language models, and learner outcomes.
	Kulkarni et al. [24]	Towards Pedagogy-Grounded Conversational AI Tutors for Interest-Based Learning	Pedagogy-grounded conversational AI tutors for interest-based learning.

4. RAG and reducing hallucination	Levonian et al. [5]	Retrieval-augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference	RAG for maths question answering, and the trade-off between groundedness and human preference.
	Németh et al. [6]	Exploring the use of retrieval-augmented generation models in higher education: A pilot study on artificial intelligence-based tutoring	Pilot study of RAG-based AI tutoring in higher education.
	Chen et al. [8]	Comparing RAG and GraphRAG for Page-Level Retrieval Question Answering on Math Textbook	Comparison of RAG and GraphRAG for page-level retrieval from a maths textbook.
	Chen et al. [11]	Mitigating Artificial Intelligence Hallucinations in Education: A Comparative Study of Retrieval-Augmented Generation (RAG) and Large Language Models	Comparison of RAG and plain language models for reducing hallucinations in education.
5. Open-source local AI deployment	Stefanko & Lang [25]	A Pipeline for Source-Grounded Learning Object and Item Generation from PDF Materials Using Large Language Models	Pipeline for generating source-grounded learning objects and items from PDF materials.
	Rao et al. [7]	Benchmarking retrieval strategies for curriculum-aligned retrieval-augmented generation using NCERT-RAG-Eval STEM textbooks	Benchmark of retrieval strategies for curriculum-aligned RAG on NCERT STEM textbooks.

2.2 Domain and theoretical foundations

2.2.1 Narrative learning in STEM

Research on science learning suggests that students understand and remember information better when it is presented as a story than when it is presented as separate, abstract facts[1], [26]. Abstract topics in secondary science, such as chemical bonding, Newton's laws of motion or cellular respiration, can be hard for a student who is new to them. When these topics are taught only through textbook prose, students can lose interest.

Research in science education shows that stories and accounts of scientific discovery can help students reach these topics [12], [26]. Stories also raise situational interest, which is the short-term attention and engagement a student feels during a lesson [15], [27].

Experimental and classroom studies report better recall and understanding when science is taught through stories [1], [2], [14]. For example, Wadasen and Lubrica taught covalent bonding through narrative fiction written by the teacher. They reported higher achievement, retention and interest than in a group taught in the usual way [13]. Agyei reported higher motivation among students who learned genetics through digital storytelling [16].

2.2.2 Student profiling and machine learning classification

A story is more likely to help when it fits the student. A single version of a lesson cannot suit every student in a class, because students differ in their backgrounds, career aspirations and preferences.

To personalise at scale, adaptive learning systems use educational data mining and supervised machine learning to build a profile of each student automatically [9], [20]. Established profiling models, such as the Felder–Silverman Learning Style Model (FSLSM), place learners along set dimensions using survey answers or records of their behaviour[18], [19]. Machine learning algorithms such as Naïve Bayes, Decision Trees and Random Forests are commonly used to classify these profiles [4], [21].

Profiling has also been applied to story-based learning. Lee et al. identified distinct narrative profiles among students in a story-based learning environment [11]. Min et

al. personalised the sequence of assessments inside such an environment [28]. Barata et al. predicted student profiles early from performance and gaming preferences [29], and Tang and Hare used a Random Forest to personalise learning in a narrative game [17]. This research applies the same approach to story preference. A Random Forest classifier places each student in one of five story themes, and Chapter 4 describes how.

2.2.3 Information retrieval and retrieval-augmented generation

Large language models are trained on broad text from the internet. They write fluently, but they are not guaranteed to be accurate on a specific, exam-oriented school syllabus. A model used without constraints can hallucinate, which means it produces claims that sound correct but are wrong, or uses terms from outside the syllabus [5], [11].

Retrieval-augmented generation (RAG) reduces this risk by joining the language model to a retrieval step, so that the model writes from retrieved source text [6]. It works in three steps.

1. Chunking and embedding. The textbooks are split into chunks of related text, and an embedding model converts each chunk into a vector [25].
2. Indexing and similarity search. The vectors are stored in a vector database. When a student requests a topic, the query is converted into a vector in the same way. The closest textbook chunks are then retrieved using a measure such as cosine similarity or Euclidean distance [8].
3. Prompting with the retrieved context. The retrieved passages are placed in the prompt, and the model is instructed to write only from them [7].

2.2.4 Lesson structure: separating the explanation from the story

Combining storytelling with strict textbook retrieval creates a trade-off. Too much story can hide the scientific definition, while text that follows the textbook too closely can lose the student's interest. Levonian et al. found a similar trade-off in maths

question answering, between how closely an answer followed the source and how much readers preferred it [5].

Recent work on AI tutors argues that prompts should be designed around teaching goals as well as the student's interests [3], [24]. Other work uses language models to personalise stories for learners [10], [22], [23].

This research handles the trade-off by splitting each lesson into two parts: a plain explanation taken from the textbook, followed by a separate story matched to the student's interest. Chapter 4 describes this structure in detail.

2.3 Related research

2.3.1 Empirical studies of narrative learning in STEM

Several studies have measured the effect of stories on achievement, retention and motivation in science.

Arya [26] compared a discovery narrative with a conventional expository text among 209 middle school students in nine classrooms. Students found the narrative more interesting. They also retained the key concepts better over the long term, especially those with little prior knowledge of the topic. Arya and Maul [1] reported an experimental study of discovery narratives in middle school science, with better understanding and recall among students who read them.

Similar work exists across science subjects. Datt and Poovaiah [2] studied a historical narrative approach to secondary school science content in a school in Mumbai, and measured its effect on memory recall. Maharaj-Sharma [14] used storytelling to teach a topic in physics. [13] taught covalent bonding through narrative fiction written by the teacher, and reported higher achievement, retention and interest than in a group taught in the usual way. Csikar and Stefaniak [12] described strategies for using storytelling in the biology classroom. [16] ran a quasi-experimental study of digital storytelling and reported higher motivation among students learning genetics. At undergraduate level, Tobler et al. [15] examined how affective and cognitive variables shape learning when biology is taught with narratives. McGee et al. [27] studied how the genre of a text affects students' interest in science.

2.3.2 Machine learning for student profiling and classification

Researchers use machine learning to model student profiles and adapt learning to them[9], [20].

Some of this work is set in story-based learning. Lee et al. [11] investigated student narrative profiles in a story-based learning environment. Min et al. [29] used collaborative filtering to personalise the sequence of assessments inside such an environment. Barata et al. [29] predicted student profiles early from performance and gaming preferences.

Other work classifies learning styles under the Felder–Silverman Learning Style Model (FSLSM). Shrestha and Pokharel [18] studied how to determine learners' style preferences. Ikawati et al. [19] used an ensemble tree method to predict learning styles from student behaviour. Wanniarachchi and Premadasa [29] used machine learning to identify students' learning styles. Garonga and Tanduk [21] compared Naïve Bayes, Decision Tree and Random Forest classifiers on students' learning styles. In their study Naïve Bayes was the most accurate at 75%, and Random Forest was the least accurate at 59%.

The closest study to this research is by Tang and Hare [17]. They built a Random Forest model into a narrative game that teaches sequential logic design. The model classifies each student by their perceived knowledge of the subject, and the game then offers individual help such as tutorials and hints. Their classifier predicts how much a student knows. The classifier in this research predicts which kind of story a student prefers.

2.3.3 Story generation with large language models

Large language models make it possible to personalise a story automatically, without a teacher writing each version by hand. Yunusov et al. [22] generated personalised narratives with language models that reflect the diversity of readers. Ruskov [23] used language models to personalise the story of a programming robot for preschool children.

In education, Lim Jia Yin et al. [3] studied the personalisation of learning content with language models. Cheng et al. [10] studied interactive narratives mediated by a language model and their effect on learner outcomes. Kulkarni et al. [24] argued that conversational AI tutors for interest-based learning should be grounded in pedagogy.

2.3.4 Retrieval-augmented generation for curriculum grounding

A language model used without constraints can give answers that are wrong or outside the syllabus. Levonian et al. [5] applied RAG to maths question answering and found a trade-off between how closely an answer followed the source and how much readers preferred it.

Other studies compare RAG with unaided models. Chen et al. [11] compared RAG with plain language models for reducing hallucinations in education. Németh et al. [6] ran a pilot study of RAG-based tutoring in higher education.

Some studies look at the retrieval step itself. Chen et al. [11] compared embedding-based RAG, keyword search and GraphRAG on 477 questions from an undergraduate mathematics textbook. Embedding-based RAG found the correct page more often than GraphRAG. RAG also improved answer quality over no retrieval, and the gain was larger for a weaker open-source model. [25] built a pipeline that generates source-grounded learning objects and items from PDF materials. Rao et al. [7] benchmarked retrieval strategies for curriculum-aligned RAG on NCERT STEM textbooks.

2.4 Existing systems products and techniques

This section compares the Narrative Learning component with existing systems and products, and then lists the established techniques it is built on.

2.4.1 Existing systems and products

The existing systems fall into three groups.

The first group is story-based learning environments in which the story is written in advance. Crystal Island [30] is a narrative game for eighth-grade microbiology. The student plays a visitor to an island and solves a mystery illness by applying what they learn, and the story follows a fixed branching structure. Tang and Hare [17] added a

Random Forest model to a narrative game that teaches sequential logic design. The model classifies each student by their level of knowledge, and the game then offers suitable help such as tutorials and hints.

The second group uses large language models to generate personalised content. MirrorStories [22] generates short stories that reflect the reader's name, gender, age, ethnicity and interests. Readers rated its personalised stories as more engaging than generic ones. The stories teach a moral and are not tied to a curriculum. PAGE [28] rewrites university course material to fit each student's major, year of study, interests and learning preferences, with supporting material retrieved by web search. In a study with 40 university students, the personalised group scored higher on a knowledge test. PAGE adapts examples and analogies and does not write stories. Oak Story [5] is a mobile app that runs an interactive story about oak woodland ecology for students in Grades 4 to 6. Its learning objectives come from a national science standard. In a study with 47 students, the group using the adaptive story showed larger learning gains than the control group.

The third group is commercial AI tutors. Khanmigo [31] is a paid AI tutor from Khan Academy, built on GPT-4 and tied to Khan Academy's own content library. It guides a student towards an answer through conversation. It does not teach through stories, and its content is not the Sri Lankan syllabus.

Table 2:Comparison of existing systems with this research

System	Teaches through a story	Personalised by	Source of facts	Model
Crystal Island [Rowe et al.]	Yes, written in advance	Not in the base game	Written by its designers to a state science standard	No language model
Narrative game [Tang & Hare]	Yes, written in advance	Student's level of knowledge	Game content on logic design	Random Forest, no language model
MirrorStories [Yunusov et al.]	Yes, generated	Name, gender, age, ethnicity, interest	None; teaches a moral	GPT-4, Claude 3 Sonnet,

				Gemini 1.5 Flash
PAGE [Lim et al.]	No; adapts examples and analogies	Major, year of study, interests, learning preferences	Course material plus web search	DeepSeek-R1
Oak Story [Cheng et al.]	Yes, generated and interactive	Student's replies during the story	Objectives from a national science standard	GPT-4o
Khanmigo [Khan Academy]	No	Conversation with the student	Khan Academy content library	GPT-4
This research	Yes, generated	Story theme predicted from interest and career aspiration	Official Grade 10–11 Science textbook, through retrieval	Llama 3.1, run locally

Among the systems reviewed, none combines a generated story, a student profile predicted by a classifier, and facts limited to a school textbook. The systems that generate content rely on commercial cloud models, except PAGE, which uses DeepSeek-R1.

2.4.2 Techniques used in this research

The component is built from five established techniques. Table 2.3 lists each one with its role in this research.

Table 3:Established techniques used in this research

Technique	**What it does**	**Role in this research**	**Source**
Random Forest	Combines many decision trees and takes their majority vote	Predicts the student's story theme	[32]
Sentence embeddings	Turn a passage of text into a vector, so that passages with similar meaning are close together	Represent the textbook passages and the student's topic	[33]
Vector similarity search	Finds the stored vectors closest to a query vector	Retrieves the textbook passages for a topic	[34]

Retrieval-augmented generation	Gives retrieved passages to a language model so that it writes from them	Keeps the lesson tied to the textbook	[35]
Open-weight language model	A model whose weights are published, so it can run on a local machine	Writes the lesson	[36]

2.5 Gap synthesis and conceptual framework

The literature shows that stories help students learn science, and that retrieval-augmented generation keeps a language model's answers closer to a source text. The systems reviewed in Section 2.4 do one or the other. None of them generates a story for an individual student while taking its facts from a school textbook. Within this, three specific gaps remain.

The first gap is how the student is profiled. Existing classifiers predict a student's level of knowledge [17] or learning style [4], [21]. Systems that generate personalised content take the student's attributes directly from a form or a conversation [3], [22] None of the reviewed systems uses a classifier to predict which kind of story a student prefers, and then uses that prediction to guide the writing.

The second gap is the source of facts and where the model runs. The story systems that use a language model rely on commercial cloud models and do not retrieve from a textbook [10], [22]. Studies that do retrieve from textbooks focus on question answering, not stories [8]. None of the reviewed systems works from the Sri Lankan Grade 10–11 Science textbooks, and none runs story generation on a local model. Chen et al. also found that retrieval improved answers more for a weaker open-source model than for a commercial one [8]. This suggests that retrieval can make up part of the difference when a smaller local model is used.

The third gap is lesson structure. Levonian et al. found a trade-off between staying close to the source and being preferred by readers [5]. A story that carries the explanation inside it faces the same trade-off. None of the reviewed systems describes separating a plain explanation of the science from the story that follows it.

Table 4 sets out each gap, how this research responds, and the research question that tests it.

Table 4:Research gaps and the response of this research

Gap	What the literature has	What this research adds	Tested by
Student profiling	Classifiers for knowledge level and learning style; personalisation from stated attributes	A classifier that predicts a story theme from interest and career aspiration	RQ1
Source of facts and model	Textbook retrieval for question answering; story generation on cloud models with no textbook	Stories written from retrieved passages of the official textbook, on a local model	RQ2
Lesson structure	One block of text that mixes the story and the explanation	A plain explanation followed by a separate story	RQ3

The conceptual framework of this research joins these three responses in one sequence. A student's interest and career aspiration go into a classifier, which gives a story theme. The topic the student selects is used to retrieve passages from the official textbook. The theme and the passages together guide a language model, which writes a plain explanation followed by a story. The framework expects three outcomes, one for each research question. The theme matches the student's story preference. The lesson stays faithful to the textbook. The student understands the topic better, and finds it more interesting, than with the textbook alone.

3 RESEARCH METHODOLOGY

3.1 Research design

This research follows a design-and-evaluate approach. I built a working system, the Narrative Learning component, and then tested it against the three research questions. The evaluation is quantitative. It uses classifier accuracy, checks of the generated stories against the textbook, and quiz scores and ratings from students.

The work was carried out in four phases.

1. Problem and requirements. I reviewed the literature and the Grade 10–11 Science textbooks, and defined the two research problems: stories that are not grounded in the textbook, and personalisation that does not scale.
2. Design. I designed the three parts of the component: the classifier that predicts a story theme, the retrieval of textbook passages, and the two-part lesson format. Chapter 4 describes the design.
3. Implementation. I trained the classifier on the survey data, indexed the textbooks, and built the component into the group's web application. Chapter 5 describes the implementation.
4. Evaluation. I evaluated the component in four ways, as shown in Table 3.1. Chapter 6 reports the results.

Table 5:How the component is evaluated

Evaluation	Answers	Method	Main measures
Classifier tests	RQ1	Stratified cross-validation on the survey data, and 12 benchmark personas with expected themes	Accuracy and a confusion matrix
Lesson generation benchmark	RQ2	12 test cases run with Llama 3.1 and with a baseline model	Quality score, valid output rate, textbook similarity, story length, generation time
Non-functional tests	System quality	15 automated checks on the running component	Number of checks passed

Comparison study	RQ3	Two groups: one receives the story lesson and the other reads the textbook	Quiz score and interest rating

3.2 Data participants or study materials

This research uses three sets of material: a student survey for the classifier, the official textbooks for retrieval, and a group of students for the comparison study.

3.2.1 Student survey

The participants are Sri Lankan students in Grades 10 and 11 who study Science in English and are preparing for the G.C.E. O/L examination. I collected their answers with an anonymous Google Form. The form asked six multiple-choice questions:

- the student's grade
- what they enjoy most in their free time
- the job they hope to have
- how school science feels for them, coded as a low, medium or high level of struggle
- the style of story they would want to read
- the story opening that would make them keep reading

A total of 28 students completed the survey, 13 in Grade 10 and 15 in Grade 11..

Before training, I shortened each answer option to a single value. For example, "Mobile gaming (PUBG, Free Fire, eSports)" became "Gaming". Of the 26 responses, 26 were used for training. [The two responses that gave "Other" as the hoped-for job were left out.]

Because the sample is small, I added 64 synthetic student records, which gave 90 training records in total. The real responses were given three times the weight of the synthetic records, so that the real data is not drowned out. The label for each record is the student's preferred story style, which corresponds to one of the five story themes.

Table 6 shows the number of records for each theme

Table 6:Training records for each story theme

Story theme	Real responses	Synthetic records	Total
Action & Competition	3	12	15
Creative Expression	5	23	28
Sci-Fi & Innovation	7	16	23
Nature & Discovery	5	8	13
Logic & Mystery	6	5	11
Total	26	64	90

3.2.2 Textbooks

The source of facts is the official English-medium Science pupil books and teacher guides for Grades 10 and 11.

The text was extracted from the PDFs and split into passages of about 200 to 400 words, each tagged with its grade and topic. The passages are stored in two indexes: the component's own index, and a shared index used by the whole project. Passages that contain only a header or a page number are filtered out before they reach the language model. Chapter 5 describes the indexing in detail.

3.3 Experimental or study procedure

The study had three procedures, one for each research question.

3.3.1 Procedure for the classifier (RQ1)

1. I collected the survey responses with the Google Form described in Section 3.2.1.
2. I shortened each answer option to a single value and encoded the interest, the hoped-for job and the story theme as numbers.
3. I combined the 26 real responses with the 64 synthetic records, and gave each real response three times the weight of a synthetic record.
4. I trained a Random Forest classifier to predict the story theme from two inputs, the interest and the hoped-for job.

5. I tested the classifier with stratified cross-validation, using [number] folds. In each fold, the classifier was trained on part of the data and tested on the rest.
6. I recorded the accuracy and a confusion matrix.
7. After a student received a theme in the live system, they were asked whether the theme matched them. I recorded these yes or no answers as a second check.

3.3.2 Procedure for the stories (RQ2)

1. I prepared 12 test cases. Each case has a topic from the textbook, a student persona, an expected story theme and a question. The persona gives the interest, the hoped-for job and the level of struggle. An example question is "Explain friction in simple terms."
2. I chose the topics to cover both grades and all four textbook parts.
3. For each case, the system predicted the theme, retrieved the textbook passages and generated the lesson with Llama 3.1.
4. A test script recorded six values for each lesson: the rubric score, whether the output was in the required format, the similarity to the textbook passages, the number of words in the story, whether the story had three paragraphs, and the time taken.
5. I ran the same 12 cases again with a baseline model, Llama 2 7B, and recorded the same values.
6. I compared the two sets of results.

3.3.3 Procedure for the comparison study (RQ3) – pending

1. [Number] students took part. They were split into two groups of [number].
2. Both groups studied the same topic, [topic].
3. The first group received the story lesson from the system. The second group read the same topic in the textbook.

4. Both groups had [number] minutes.
5. Both groups then took the same quiz of [number] questions on the topic.
6. Both groups answered a rating question on how interesting they found the lesson, on a scale of 1 to 5.

3.3.4 Procedure for the non-functional tests

1. I started the component on the local machine.
2. I ran an automated suite of 15 checks against it.
3. Each check sent a request to the component or inspected a file, and compared the result with an expected value or a time limit.
4. For theme classification, the suite timed 8 requests and recorded the mean and the 95th percentile.
5. I recorded each check as passed or failed.

3.4 Measures and evaluation metrics

The component was evaluated in four ways: tests of the classifier, a lesson generation benchmark, non-functional tests, and a comparison study with students. Table 3.3 lists the measures used in each.

Table 7:Measures used in the evaluation

Evaluation	Measure	How it is calculated
Classifier (RQ1)	Cross-validation accuracy	Number of students given the correct theme, divided by the number tested
Classifier (RQ1)	Confusion matrix	A table of the real theme against the predicted theme, for all five themes
Classifier (RQ1)	Benchmark theme accuracy	Number of benchmark cases where the predicted theme matches the expected theme, divided by 12
Lesson benchmark (RQ2)	Generation quality score	A rubric score from 0 to 100 for each lesson
Lesson benchmark (RQ2)	Valid output rate	Number of lessons returned in the required format, divided by 12

Lesson benchmark (RQ2)	Textbook similarity	How close the lesson is in meaning to the source textbook passages, as a percentage
Lesson benchmark (RQ2)	Story length and structure	Number of words in the story, and whether it has three paragraphs
Lesson benchmark (RQ2)	Generation time	Seconds taken to produce one lesson
Non-functional tests	Tests passed	Number of automated checks passed, out of 15
Comparison study (RQ3)	Quiz score	Number of quiz questions answered correctly
Comparison study (RQ3)	Interest rating	The student’s rating of how interesting the lesson was, from 1 to 5

The classifier is measured in two ways. Cross-validation accuracy and the confusion matrix come from the survey data. Benchmark theme accuracy comes from the 12 benchmark cases, where each case has an expected theme set by [how the expected themes were chosen].

The lesson generation benchmark has 12 test cases. Each case combines a topic from the Grade 10 or 11 Science textbook, a student persona and a question. The persona gives the interest, the hoped-for job and the level of struggle. The generation quality score comes from a rubric that checks [list the rubric items, for example whether the concept is present, whether it is explained, whether the story has three paragraphs and whether equations are included]. The target story length is about 200 to 300 words in three paragraphs. The same 12 cases were also run with a baseline model, Llama 2 7B, so that the two models can be compared.

The non-functional tests are 15 automated checks in three areas. The first is response time, with a limit of 500 milliseconds for the status check and 800 milliseconds for theme classification. The second is configuration, such as the survey loading with six questions and the classifier returning one of the five themes. The third is privacy, which checks that responses contain no keys and that the settings file and textbook index are kept out of the code repository.

For the comparison study, the quiz score measures understanding and the interest rating measures engagement. Both are compared between the group that received the story lesson and the group that read the textbook.

3.5 Data analysis

The data from each evaluation was analysed with descriptive statistics. The analysis was done in Python with the pandas and scikit-learn libraries.

Classifier. Cross-validation gives one accuracy figure for each fold. I report the mean of these figures as the accuracy of the classifier. I then read the confusion matrix to find which themes the classifier predicts well and which pairs of themes it confuses. For the 12 benchmark personas, I count how many predicted themes match the expected theme.

Lesson generation benchmark. For the quality score, the textbook similarity, the story length and the generation time, I report the mean and the range across the 12 cases. For the valid output rate and the three-paragraph check, I report the number of cases that passed. I then place the results for Llama 3.1 and the baseline model side by side, measure by measure.

Non-functional tests. I report the number of checks passed out of 15. For the timed checks, I compare the measured time with its limit.

Comparison study. For each group, I report the mean quiz score and the mean interest rating. I then compare the two groups. Because the groups are small, the differences are reported as an indication and are not tested for statistical significance.

3.6 Ethics, privacy, security and responsible AI

Ethics. The survey participants are school students in Grades 10 and 11. The survey was anonymous. It did not ask for a name, a school or any contact detail, so no response can be linked to a student. The answers are used only for this research and are reported only as totals. Because no personal data was collected, the survey carried a low risk to the students. Students who take part in the comparison study do so by choice, and taking part has no effect on their school marks.

Privacy. The component stores only the survey answers and the story theme. It does not store names. The language model runs on the local machine, so the text a student enters is not sent to an outside AI service. An automated test confirms that the component's responses contain no keys or vendor names.

Security. Keys and settings are kept in a settings file that is excluded from the code repository. The textbook index is excluded as well. Both exclusions are checked by automated tests.

Responsible AI. A language model can produce wrong statements, so the component limits this risk in four ways. The model is instructed to write only from passages retrieved from the official textbook. If the model fails to produce a usable lesson, the component shows a fallback lesson built from the textbook text, not an invented one. Each lesson shows the textbook passages it was written from, with the file, chapter and page, so that a student or teacher can check it. The lessons are presented as study aids that support the teacher. They are not official answers for the examination.

The training data for the classifier includes synthetic records. This is stated openly in Section 3.2.1 and in the limitations. The use of AI tools in writing this thesis is declared in Appendix B.

3.7 Reproducibility and research artefacts

The study can be repeated with the artefacts in Table

Table 8:Research artefacts

Artefact	Description	Where it is kept
Source code	The component, the training script and the user interface	The group's code repository
Survey instrument	The six questions and their answer options	Appendix A
Survey responses	28 anonymous responses	The project's shared evidence folder, with a summary in Appendix A
Training data	90 records, each marked as real or synthetic	The project's shared evidence folder; rewritten each time the training script runs
Trained classifier	The model file and the encoder file	Written by the training script, next to the component code

Textbook indexes	The component's own index and the shared project index	Shared separately; not in the repository
Benchmark	The 12 test cases and the result reports for both models	Test cases and script in the group's code repository; result reports in the shared evidence folder
Non-functional tests	The 15 checks and their result report	Test script in the group's code repository; result report in the shared evidence folder

Three things make the classifier results repeatable. The training script uses a fixed random seed. The classifier settings are fixed and are listed in Chapter 5. The library versions are also listed in Chapter 5.

The language models are public. Both Llama 3.1 and the embedding model can be downloaded through Ollama under the same names used in this research.

Two parts of the study cannot be repeated exactly. The textbook indexes and the settings file are not in the code repository, so they must be obtained separately or rebuilt from the textbook PDFs. The language model also does not give the same output every time, so a repeat of the benchmark will give scores that are close to those reported but not identical.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

The requirements for the Narrative Learning component are in two groups. Functional requirements state what the component must do. Non-functional requirements state how well it must do it.

4.1.1 Functional requirements

Table 9:Functional requirements

ID	Requirement
FR1	The component shall present a survey of six questions: grade, free-time interest, hoped-for job, level of struggle with science, preferred story style and preferred story opening.
FR2	The component shall predict one of five story themes for the student from their interest and hoped-for job, using a trained classifier.
FR3	If the classifier is unavailable or meets an answer it has not seen, the component shall choose the theme from the student's own story style and opening answers.
FR4	The component shall let the student select a topic from the Grade 10 or 11 Science textbooks, and add a note on what they find hard.
FR5	The component shall retrieve passages on the selected topic from the official textbook, and leave out passages that contain only a header or a page number.
FR6	If too little text is retrieved from the component's own index, the component shall add passages from the shared project index.
FR7	The component shall generate a lesson in two parts: a plain explanation with no characters or plot, followed by a story of about 200 to 300 words in three paragraphs.
FR8	The story shall match the student's theme and interest and be set in Sri Lanka. The depth of the explanation shall match the student's level of struggle.
FR9	If the language model returns an unusable lesson, the component shall try again, and then show a fallback lesson built from the textbook text.
FR10	The component shall display the explanation, the story, the key definitions, the equations, the exam points and the source passages with their file, chapter and page.
FR11	The component shall let the student say whether the theme matched them.
FR12	The component shall offer these functions through an interface within the group's web application.

4.1.2 Non-functional requirements

Table 10:Non-functional requirements

ID	Requirement	How it is checked
NFR1	Response time. The status check shall respond within 500 milliseconds, and theme classification within 800 milliseconds for 95% of requests.	Non-functional tests
NFR2	Privacy. The component shall not store student names, and its responses shall contain no keys or vendor names.	Non-functional tests
NFR3	Local running. The language model and the embedding model shall run on the local machine, so that student text is not sent to an outside AI service.	Design, Section 4.3
NFR4	Cost. Generating a lesson shall carry no per-token charge.	Design, Section 4.3
NFR5	Security. The settings file and the textbook index shall be excluded from the code repository.	Non-functional tests
NFR6	Grounding. The facts in a lesson shall come from the retrieved textbook passages.	Lesson generation benchmark
NFR7	Reliability. Every request shall return a lesson in the required format.	Lesson generation benchmark
NFR8	Modularity. The component shall work as a separate module, so that it can be changed without affecting the other three modules.	Integration, Section 5.3

4.2 Component architecture within the complete solution

This section first describes the complete solution built by the group, then shows where the Narrative Learning component sits within it, and then describes the internal architecture of the component.

4.2.1 The complete solution

The complete solution is a single web application with four learning modules. A student opens it in a web browser and chooses a module. Figure shows the structure.

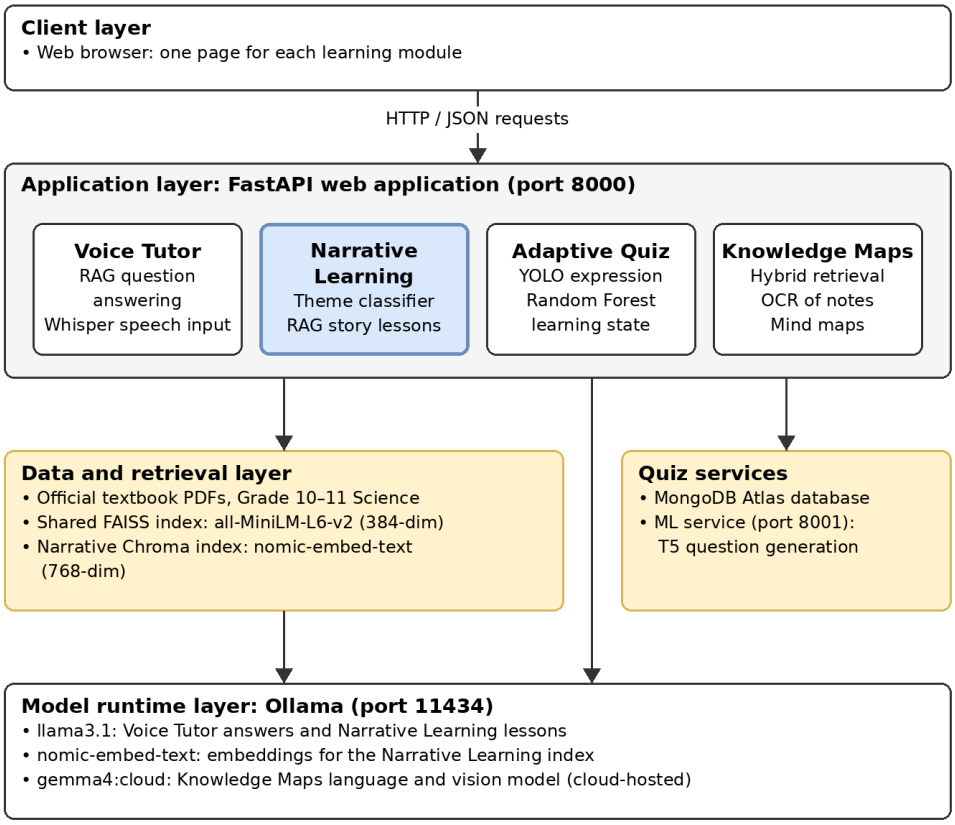


Figure 2:Architecture of the complete solution, with the Narrative Learning component highlighted

The four modules are:

- **Voice Tutor** answers typed or spoken questions from the syllabus. It retrieves textbook passages, transcribes speech with Whisper, and can read the answer aloud.
- **Narrative Learning** matches a student to a story theme and teaches a topic through a two-part lesson. This is my individual component.

- **Adaptive Quiz** is a multi-level quiz that tracks response time, answer changes and facial expression, and combines them in a Random Forest model of the student's learning state. It can also generate questions from a PDF with a pre-trained T5 model.
- **Knowledge Maps** checks a student's notes against the syllabus, reads handwritten notes, builds mind maps and suggests videos.

The modules are built on a set of shared resources. The official textbook PDFs are the source of facts. A shared textbook index, built from those PDFs with FAISS, lets a module search the textbooks by meaning. The Ollama runtime serves the language models. The Adaptive Quiz also has its own database and its own machine learning service.

4.2.2 The place of Narrative Learning in the solution

Narrative Learning is one of the four modules inside the web application. It has its own page in the student interface and its own set of requests under the address /narrative-learning.

The component shares three things with the rest of the solution. It runs inside the same web application, so it starts and stops with the other modules. It uses the same Ollama runtime as Voice Tutor, so the group maintains only one installation of the language models. It also uses the shared textbook index as a backup when its own index returns too little text.

The component owns four things that no other module uses. These are the theme classifier, its own textbook index, the lesson generation engine, and its page in the student interface.

The component does not call the other modules directly. This keeps it separate, so that its classifier can be retrained or its index rebuilt without affecting Voice Tutor, Adaptive Quiz or Knowledge Maps.

4.2.3 Internal architecture of the component

Figure 3 shows the parts of the component and how they connect.

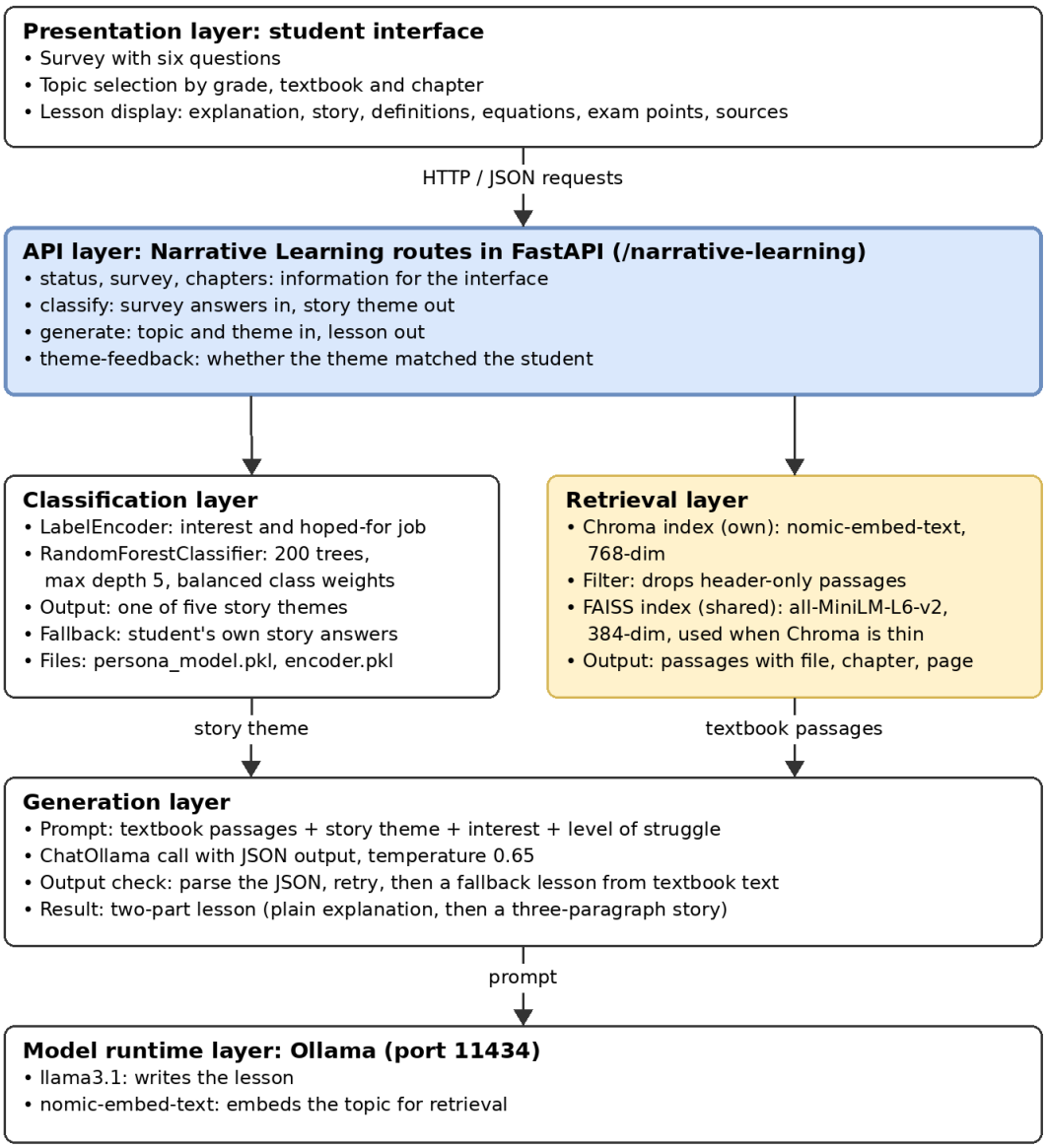


Figure 3:Internal architecture of the Narrative Learning component

The component has five parts.

Student interface. This is the page the student sees. It presents the survey, lets the student select a textbook topic, and displays the finished lesson.

Component API. The interface talks to the component through a small set of requests, listed in Table 11. The API passes each request to the right part of the component and returns the result.

Theme classifier. This part predicts the student's story theme. It encodes the student's interest and hoped-for job as numbers and passes them to the trained Random Forest. If the trained classifier is not available, or the student gives an answer it has not seen, this part chooses the theme from the student's own story style and opening answers. It reads the classifier and the encoder from two saved files.

Retrieval. This part finds the textbook passages for the selected topic. It searches the component's own index first. It removes passages that contain only a header or a page number. If too little text remains, it adds passages from the shared project index.

Lesson generation. This part builds the prompt from the retrieved passages, the story theme, the student's interest and their level of struggle. It sends the prompt to Llama 3.1, which runs in the Ollama runtime. It then checks the output. If the output is not usable, it tries again, and if that also fails it returns a fallback lesson built from the textbook text.

Table 11:Requests handled by the component

Request	What it receives	What it returns
Status	Nothing	Whether the component is ready, and whether the trained classifier is loaded
Survey	Nothing	The six survey questions and their options
Classify	The student's survey answers	The story theme
Chapters	Nothing	The textbooks and their chapters
Generate	The topic, the theme, the interest and the level of struggle	The lesson: explanation, story, key definitions, equations, exam points and source passages
Theme feedback	Whether the theme matched the student	A confirmation

4.3 Design alternatives and decisions

Five design decisions shaped the component. For each one, this section gives the alternatives I considered, the choice I made and the reason for it. Table 4.5 at the end summarises them.

4.3.1 The classifier

I considered three kinds of classifier: a neural network, simpler classifiers such as Naïve Bayes or a single Decision Tree, and a Random Forest.

I chose a Random Forest. The survey answers are categories, not numbers, and the data set is small. A Random Forest handles both well, trains on an ordinary computer without a graphics card, and can report which input mattered more. A neural network needs far more data than 90 records. I limited each tree to a depth of 5 so that the forest could not memorise the small data set, and I balanced the class weights because the themes have different numbers of records.

This choice has a weakness that should be stated. Garonga and Tanduk compared the three classifiers on student learning styles and found Naïve Bayes the most accurate and Random Forest the least [21]. Their tasks and data are different from mine, so their result does not decide the choice here.

Two further decisions concern what the classifier learns. The label is the student's preferred story style, not their interest, because a hobby is not the same as a taste in stories. The inputs are the interest and the hoped-for job only. The level of struggle is left out of the classifier and is used later to set the depth of the explanation.

4.3.2 The textbook indexes

I considered three options: a cloud vector database, a single local index, and two local indexes.

I chose two local indexes. A cloud database would add cost and send the textbook queries outside the machine. The component's own index is built with Chroma, which stores the file name with each passage, so a search can be limited to the textbook the student selected.

The second index was added because of a problem found during development. Many passages in the first version of the component's index contained only a page header,

while the shared project index held the body text of the same pages. Using the shared index as a backup solved this without rebuilding everything. The component also filters out header-only passages before they reach the language model.

4.3.3 The language model and where it runs

I considered a paid cloud model and a model that runs on the local machine.

I chose a local model, run through Ollama. A local model has no per-token charge, which matters for a school. The student's text stays on the machine. Voice Tutor already uses the same runtime, so the group maintains one installation.

I then chose between two local models by testing both on the same 12 benchmark cases. Table 4.6 shows the results. Llama 3.1 scored higher on quality, always returned output in the required format, always wrote three paragraphs and was faster. I therefore selected Llama 3.1.

Table 12:Benchmark comparison of the two local models

Measure	**Llama 3.1**	**Llama 2 7B**
Generation quality score	91.7%	63.9%
Output in the required format	12 of 12	8 of 12
Similarity to the textbook	78.0%	62.4%
Stories with three paragraphs	100%	8%
Average story length	230.9 words	158.4 words

4.3.4 The structure of the lesson

I considered one continuous story that carries the explanation inside it, and a lesson in two separate parts.

I chose two parts: a plain explanation first, then the story. In an exam-oriented subject, the student needs to find the law and its exact wording quickly. A law buried inside a plot is hard to revise from. Levonian et al. found a related trade-off, between staying close to the source and being preferred by readers [5]. Separating the two parts lets the explanation stay close to the textbook while the story is free to be engaging.

4.3.5 Handling the model's output

I considered accepting the model's reply as free text, and requiring a fixed format.

I chose a fixed format. The interface shows the explanation, the story, the definitions, the equations, the exam points and the sources in separate places, so the reply must arrive in separate fields. The model is asked for JSON output. A local model does not always follow the format, so the component checks each reply. If the reply is unusable, the component asks again. If that also fails, it shows a fallback lesson built from the textbook text. The fallback never contains an invented story, so a failure cannot introduce wrong facts.

Table 13:Summary of design decisions

Decision	Alternatives	Choice	Main reason
Classifier	Neural network, Naïve Bayes, Decision Tree, Random Forest	Random Forest	Suits small, categorical data and trains without special hardware
Textbook indexes	Cloud database, one local index, two local indexes	Two local indexes	The shared index fills gaps left by header-only passages
Language model	Paid cloud model, local model	Local model through Ollama	No per-token cost, and student text stays on the machine
Local model	Llama 2 7B, Llama 3.1	Llama 3.1	Higher quality, valid output every time, and faster
Lesson structure	One continuous story, two separate parts	Two separate parts	The law stays easy to find and close to the textbook

Output handling	Free text, fixed format	Fixed format with retry and fallback	The interface needs separate fields, and failures must not add wrong facts

4.4 Detailed design

This section gives the detailed design of the component: its data, the classifier, retrieval, the prompt, the handling of the model's output, and the order of events in one session.

4.4.1 Data design

The component works with three sets of data.

The first is the student's survey answers. Table 14 lists the six fields.

Table 14:Survey answer fields

Field	Values	Used for
grade	Grade 10 or Grade 11	Selecting the textbook
interest	13 values, such as Gaming, Cricket and Coding/ICT	Classifier input, and the story
aspiration	9 values, such as Engineer, Teacher and Artist	Classifier input, and the story
struggle_level	Low, Medium or High	Depth of the explanation
story_style	Five options, one for each story theme	Training label, and the fallback
story_opening	Five options, one for each story theme	The fallback

The second is the training record. Each record holds the interest, the aspiration, the struggle level, the story theme and a marker that shows whether the record is real or synthetic.

The third is the lesson that the component returns. Table 15 lists its fields.

Table 15:Lesson output fields

Field	Content
science_intro	The plain explanation, called TELL
story	The three-paragraph story, called SHOW
key_definitions	The definitions the student should know
key_equations	The equations for the topic
exam_bullets	Short points for exam revision
quiz_topic	A topic for follow-up questions
sources	The textbook passages used, with file, chapter and page

4.4.2 Classifier design

The classifier is trained in five steps. The training script reads the survey responses and the synthetic records. It shortens each answer option to a single value. It encodes the interest, the aspiration and the theme as numbers. It gives each real response a weight of 3 and each synthetic record a weight of 1. It then trains the Random Forest with the settings in Table 16 and saves the classifier and the encoder as two files.

Table 16:Classifier settings

Setting	Value	Reason
Number of trees	200	Enough trees for a stable vote
Maximum depth	5	Stops the trees memorising a small data set
Class weights	Balanced	The themes have different numbers of records
Minimum records in a leaf	2	Avoids rules built on a single student
Random seed	42	Make training repeatable
Record weights	Real 3, synthetic 1	Keeps the real responses from being drowned out

When a student submits the survey, the component chooses the theme in this order:

1. If the trained classifier is switched on and its files load, use the classifier's prediction.
2. Otherwise, if the story style answer and the story opening answer point to the same theme, use that theme.
3. Otherwise, use the theme of the story style answer.
4. Otherwise, use the default theme, Action & Competition.

4.4.3 Retrieval design

When the student selects a topic, the component finds the textbook passages in six steps.

1. It turns the topic into a vector with the embedding model.
2. It searches its own index for the closest passages. If the student has selected a textbook, the search is limited to that book.
3. It removes passages that are too short or contain too few letters. These are usually page headers or page numbers.
4. If too little text remains, it adds passages on the same topic from the shared project index.
5. It trims the passages to a set length.
6. It passes the passages on with their file, chapter and page.

4.4.4 Prompt and lesson design

The prompt gives the language model four things: the retrieved passages, the story theme, the student's interest and the student's level of struggle. It then sets the rules in Table 17.

Table 17:Rules set by the prompt

Part	Rule
TELL	A classroom explanation only: the law, why it works, and the formulae. No characters.
SHOW	Three paragraphs: a hook tied to the student's interest, a scene where the science happens, and a bridge where the student in the story states the textbook law. At least about 220 words.
Level of struggle	High gives a simpler explanation. Low gives a more technical one.
Grounding	Use only the retrieved passages. Set the story in everyday Sri Lanka. No science-fiction gadgets unless the theme is Sci-Fi, and even then the facts come from the textbook.

The model is called with the settings in Table 18

Table 18:Language model settings

Setting	Value
Model	Llama 3.1, through Ollama
Temperature	0.65
Output format	JSON
Context size	6,144 tokens
Maximum output	1,800 tokens

4.4.5 Handling the output

The component reads the model's reply as JSON and checks it. If the story is too thin, the component asks the model again. If the model fails altogether, the component returns a fallback lesson built from the retrieved textbook text. The fallback contains no invented story.

4.4.6 Sequence of a session

Figure 4 shows the order of events when a student uses the component.

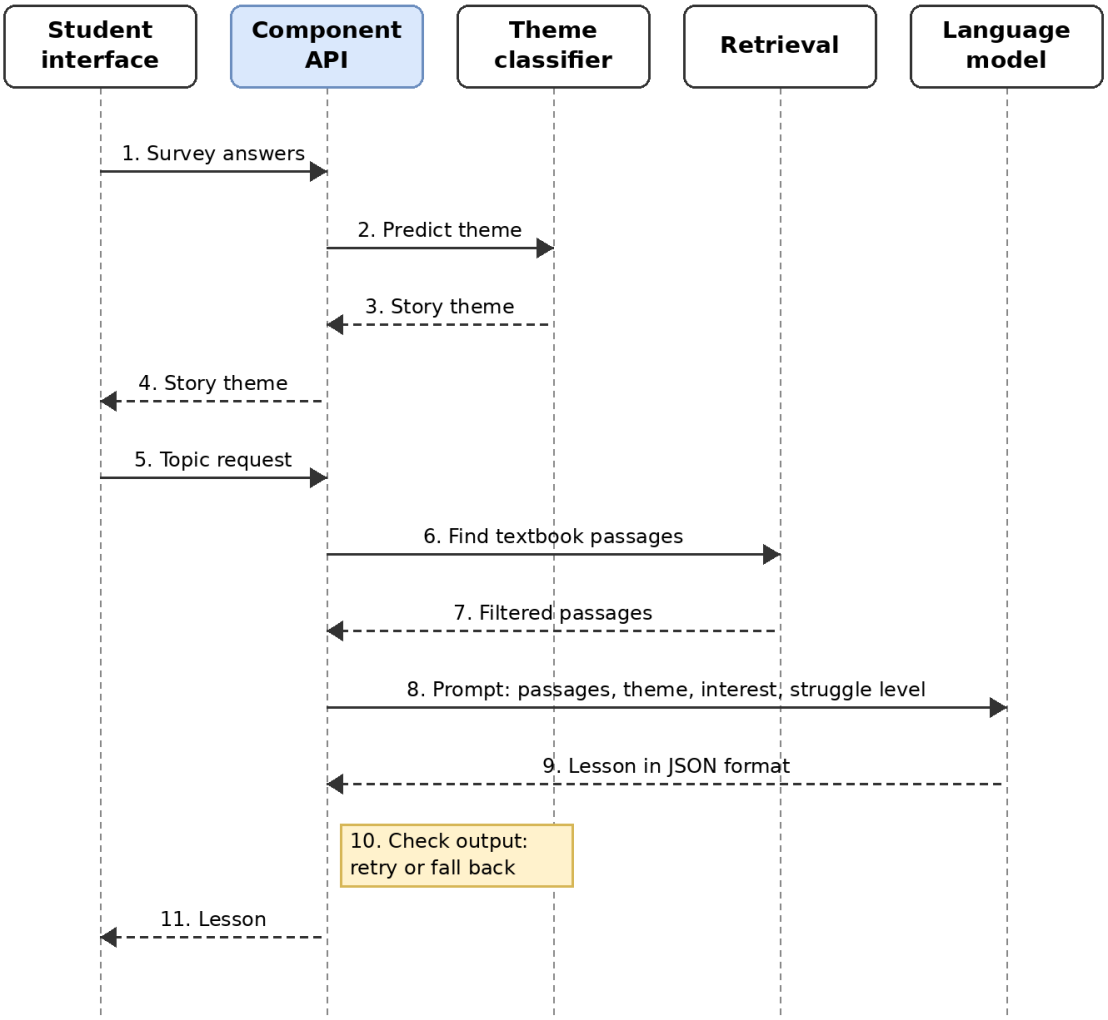


Figure 4:Sequence of events in one session

The session has two stages. In the first, the student submits the survey, the classifier predicts the theme and the interface shows it (steps 1 to 4). In the second, the student selects a topic, the component retrieves the passages and sends the prompt to the language model, checks the reply and shows the lesson (steps 5 to 11).

4.5 Standards and professional practice

I used recognised standards as guides for the design and followed common professional practice in building the component.

Table 19:Standards and practices applied

Area	Standard or practice	How it was applied
Software quality	ISO/IEC 25010 quality model	Used as a checklist for the non-functional requirements
Data protection	Sri Lanka Personal Data Protection Act, No. 9 of 2022	No personal data is collected or stored
Responsible AI	UNESCO Recommendation on the Ethics of Artificial Intelligence (2021)	Human oversight, transparency of sources, and limits on what the model may write
Coding	PEP 8 style guide for Python	Followed for naming and layout of the code
API design	REST conventions and OpenAPI	Requests are documented automatically by FastAPI
Version control	Git	Code is tracked, and secrets are kept out of the repository
Testing	Automated tests	15 non-functional checks and a 12-case benchmark

Software quality. I used the quality characteristics of ISO/IEC 25010 to decide what the non-functional requirements should cover. Functional suitability is addressed by the lesson format and the output check. Performance efficiency is addressed by the response time limits. Security is addressed by keeping keys and the textbook index out of the repository. Maintainability is addressed by building the component as a separate module. Section 4.1.2 lists these requirements, and Chapter 6 reports how each was tested.

Data protection. The component follows the principle of collecting only what is needed. The survey asks for no name, school, address or contact detail. The component stores the survey answers and the story theme without any identifier, so a stored record cannot be linked to a student. The language model runs on the local machine, so the text a student enters is not sent to an outside AI service.

Responsible AI. Three design choices follow the UNESCO recommendation. The first is human oversight: the lessons are study aids that support the teacher, and they are not official examination answers. The second is transparency: each lesson shows the textbook passages it was written from, so a student or teacher can check it. The third is limiting harm: the model may write only from retrieved textbook passages, and a failed generation falls back to textbook text. The classifier uses only the student's interest and hoped-for job. It does not use gender, school or region, because the survey does not collect them. I did not test the classifier for bias, and this is noted as a limitation.

Coding and API design. The component is written in Python and follows the PEP 8 style guide. Its requests follow REST conventions, and FastAPI produces OpenAPI documentation for them automatically. Dependencies are installed in a project virtual environment, so the component runs with known library versions.

Version control and testing. The code is kept under Git in the group's repository. The settings file that holds keys is excluded from the repository, and an automated test confirms this. The component is checked by an automated suite of 15 non-functional tests and by the 12-case lesson generation benchmark.

Academic practice. The use of synthetic training data is stated openly in Section 3.2.1, and the use of AI tools in writing this thesis is declared in Appendix B.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development environment and technology stack

The component is written in Python and runs inside the group's FastAPI web application. All of its libraries are installed in the project's virtual environment, so the versions are fixed and known. Table 20 lists the software used by the component.

Table 20:Software and library versions

Software	Version	Role in the component
Python	3.13	Programming language
FastAPI	0.141.1	Web application framework
Uvicorn	0.52.4	Web server that runs the application
scikit-learn	1.9.0	Random Forest classifier and label encoding
pandas	3.0.5	Reading and preparing the survey data
ChromaDB	1.5.9	The component's own textbook index
FAISS (faiss-cpu)	1.15.0	The shared project textbook index
sentence-transformers	6.0.0	Embeddings for the shared index
LangChain core	1.6.1	Prompt template and model calls
langchain-ollama	1.1.0	Connection to the Ollama models
langchain-chroma	1.1.0	Connection to the Chroma index
PyMuPDF	1.24 or later	Extracting text from the textbook PDFs
Ollama	llama3.1	Runs the language models on the local machine

The component uses two models, both served by Ollama. Table 20 lists them.

Table 21:Models used by the component

Model	Role
llama3.1	Writes the lesson
nomic-embed-text	Turns the textbook passages and the student's topic into vectors for the component's own index

5.1.1 Setting up the environment

The environment is set up in five steps.

1. Install Ollama and download the two models:

```
ollama pull llama3.1
```

```
ollama pull nomic-embed-text
```

2. Create the project's virtual environment and install the libraries in Table 5.1.
3. Place the settings file and the textbook indexes in the project folder. These are shared separately because they are not in the code repository.
4. Train the classifier from the survey responses:

```
python scripts/train_persona.py --survey path/to/form.xlsx
```

5. Start Ollama, then start the web application:

```
ollama serve python run.py
```

5.1.2 Configuration

The component is controlled by five settings in the settings file.

Table 22:Configuration settings

LLM_PROVIDER	ollama	Selects the local model runtime
OLLAMA_BASE_URL	http://127.0.0.1:11434	Address of the Ollama runtime on the local machine

OLLAMA_LLM_MODEL	llama3.1	The model that writes the lesson
USE_TRAINED_CLASSIFIER	1	Switches on the trained classifier
NARRATIVE_DEMO_THEME	Action & Competition	The default theme when no other choice can be made

5.2 Component implementation

The component is implemented in four parts: the classifier, retrieval, lesson generation, and the API with its interface. Table 23 shows where the code and files are kept.

Table 23:Location of the component's code and files

Item	Location
Component code	backend/components/narrative_learning/
Training script	scripts/train_persona.py
Trained classifier and encoder	persona_model.pkl and encoder.pkl, next to the component code
Component's textbook index	science_vector_db/, inside the component folder
Student interface	frontend/narrative_learning/
Survey log	data/narrative_survey.csv

The classifier. The training script reads the survey spreadsheet and the synthetic records with pandas. It shortens each answer option to one value, encodes the values with scikit-learn's LabelEncoder, and trains a RandomForestClassifier. It saves the classifier and the encoder as two files and writes the full training data to training_rows.csv. At run time, the file classifier.py loads the two files and returns a theme. If the files cannot be loaded, it falls back to the student's own story answers.

Retrieval. The component's index is a Chroma database, accessed through LangChain. The textbook passages and the student's topic are turned into vectors by nomic-embed-text, served by Ollama. Each passage is stored with its file name, so a

search can be limited to one textbook. A function checks each retrieved passage for length and for the share of letters in it, and drops those that fail. A second function adds passages from the shared project index when too little text remains.

Lesson generation. A class named StoryEngine carries out generation. It fills a LangChain prompt template with the passages, the theme, the interest and the level of struggle. It calls Llama 3.1 through ChatOllama with JSON output. It then reads the reply, asks again if the story is too thin, and builds a fallback lesson from the textbook text if the model fails.

API and interface. The component's requests are FastAPI routes under /narrative-learning. The interface page calls these routes and displays the results. Each submitted survey is added to the survey log without any name.

One problem during implementation shaped the final design. Many passages in the first version of the component's index contained only a page header. I added the passage check and the backup from the shared index to solve this, as described in Section 4.3.2.

5.3 Integration with the other components

The component is integrated with the group's solution at four points.

- **The web application.** The component's routes are registered in the group's FastAPI application, next to the routes of the other three modules. All four modules start together with one command.
- **The student interface.** The component's page sits in the shared interface folder and is reached from the same starting page as the other modules.
- **The model runtime.** The component uses the same Ollama installation as Voice Tutor, and the same Llama 3.1 model.
- **The shared textbook index.** The component reads the index that the group builds from the textbook PDFs, and uses it as a backup for its own index.

The component does not call the other modules, and they do not call it. [Each lesson returns a suggested quiz topic, which links the student to Adaptive Quiz.]

Three items are not in the code repository: the settings file, the shared index and the component's own index. The group shares these as a separate pack, so that each member can run the complete solution.

The non-functional tests confirmed the integration. With the complete application running, the component reported that it was ready, that the trained classifier was loaded, and that all four textbooks were available.

5.4 Verification and software testing

The component was tested in three ways.

Classifier validation. The training script tests the classifier with stratified cross-validation each time it trains, and prints the accuracy.

Lesson generation benchmark. A test script runs 12 fixed test cases through the complete component and records the measures described in Section 3.4. Running the same cases again after a change shows whether the change made the lessons better or worse. I used it in this way to compare the two language models.

Non-functional tests. An automated suite of 15 checks runs against the component while it is running. Table 24 lists the checks.

Table 24:Non-functional checks

Area	Checks
Availability	The status request responds, and the chapters request responds with four textbooks
Response time	The status request responds within 500 milliseconds, and theme classification within 800 milliseconds for 95% of requests
Configuration	The classifier is in trained mode. The survey loads with six questions, including interest and aspiration. Five themes are

	published, and a returned theme is one of the five. The classifier file and the encoder file are present
Privacy and security	The classification response contains no keys or vendor names. The settings file, the textbook index folder and the test folder are excluded from the repository

Table 25 shows which tests cover which requirements.

Table 25:Requirements covered by each kind of test

Test	Requirements covered
Classifier validation	FR2
Lesson generation benchmark	FR2, FR5 to FR9, NFR6, NFR7
Non-functional tests	FR1, FR2, NFR1, NFR2, NFR5
Manual testing of the interface	FR4, FR10, FR11, FR12

5.5 Deployment and operation

The component runs on a single computer together with the rest of the solution. It has not been deployed to a public server or to a school.

To start with the solution, Ollama is started first and the web application second. A student then opens the application in a web browser on the same computer and chooses Narrative Learning. The language model is kept loaded in memory after its first use, so that later lessons do not wait for it to load again.

Three tasks keep the component working over time.

- **Retraining the classifier.** When new survey responses are collected, the training script is run again and the application is restarted.
- **Rebuilding the index.** If a textbook is replaced or added, the index is rebuilt from the PDF files.
- **Checking the component.** The status request shows whether the component is ready and whether the trained classifier is loaded. The non-functional test

suite can be run at any time to confirm that the component still meets its requirements.

6 RESULTS AND VALIDATION

6.1 Research results

6.1.1 The classifier (RQ1)

I tested the classifier with stratified 5-fold cross-validation, repeated 20 times with different splits. Table 26 shows accuracy.

Table 26:Accuracy of the theme classifier

All 90 training records	76.1% (lowest 67.8%, highest 80.0%)
The 26 real responses only	56.0% (lowest 46.2%, highest 61.5%)
Always predicting the largest theme	31.1%
Random guess among five themes	20.0%
All 90 training records	76.1% (lowest 67.8%, highest 80.0%)

The classifier is clearly better than a random guess and better than always predicting the largest theme. It is right a little more than half the time on all records, and a little less than half the time on the real responses.

Table 27 shows the confusion matrix for one run. Rows are the real theme and columns are the predicted theme.

Table 27:Confusion matrix of the theme classifier

Real theme	Action	Creative	Sci-Fi	Nature	Logic	Correct
Action & Competition	7	0	5	2	1	7 of 15
Creative Expression	5	13	0	6	4	13 of 28
Sci-Fi & Innovation	2	2	8	4	7	8 of 23
Nature & Discovery	0	1	2	10	0	10 of 13
Logic & Mystery	0	2	1	0	8	8 of 11

The classifier predicts Nature & Discovery and Logic & Mystery well. It is weakest on Sci-Fi & Innovation, which it most often confuses with Logic & Mystery. It also confuses Creative Expression with Nature & Discovery and with Action & Competition.

Of the two inputs, the interest mattered more. Its importance in the trained classifier was 0.655, against 0.345 for the hoped-for job.

On the 12 benchmark personas, the predicted theme matched the expected theme in 8 cases, which is 66.7%. The four mismatches were Gaming with Engineer, Coding/ICT with Engineer, Reading with Teacher, and Photography with Artist.

The survey data explains part of the error. Students with the same interest and hoped-for job sometimes chose different story styles. Students who chose Gaming and Engineer picked three different themes. A classifier that sees only those two answers cannot separate such students.

6.1.2 The generated lessons (RQ2)

Table 28 shows the benchmark results for Llama 3.1 and for the baseline model across the 12 test cases.

Table 28:Results of the lesson generation benchmark

Measure	**Llama 3.1**	**Llama 2 7B (baseline)**
Generation quality score	91.7% (83.3% to 100.0%)	63.9% (50.0% to 75.0%)
Output in the required format	12 of 12	8 of 12
Similarity to the textbook	78.0% (71.8% to 81.9%)	62.4% (55.3% to 68.2%)
Average story length	230.9 words	158.4 words
Stories with three paragraphs	100%	8%

Llama 3.1 was better on every measure. Every one of its lessons arrived in the required format and had a three-paragraph story. Its lessons were closer in meaning to the textbook, with a similarity of 78.0% against 62.4% for the baseline. In all 12 cases, the report marked the concept as present and explained, and the retrieval step returned passages from four sources.

Table 29 gives the result for each case with Llama 3.1.

Table 29:Llama 3.1 results for each benchmark case

Topic	Persona	Theme match	Words	Similarity	Score	Time
Friction	Cricket, Athlete	Yes	321	80.4%	100.0%	52.7 s
Newton's laws of motion	Robotics, Engineer	Yes	221	76.8%	91.7%	30.2 s
Waves and their applications	Music, Artist	Yes	196	79.7%	83.3%	27.5 s
Photosynthesis	Nature, Doctor	Yes	282	80.2%	91.7%	51.5 s
Turning effect of a force	Mathematics, Scientist	Yes	213	81.2%	100.0%	50.7 s
Current electricity	Gaming, Engineer	No	193	77.9%	91.7%	31.9 s
Geometrical optics	Art, Architect	Yes	170	77.5%	91.7%	36.8 s
Electronics	Coding/ICT, Engineer	No	236	71.8%	91.7%	44.4 s
Inheritance	Reading, Teacher	No	193	79.2%	91.7%	77.0 s
Heat	Photography, Artist	No	188	81.9%	91.7%	41.2 s
Motion in a straight line	Astronomy, Scientist	Yes	264	73.3%	83.3%	38.8 s
Work, energy and power	Kandyan Dancing, Teacher	Yes	294	76.3%	91.7%	42.4 s

Two points stand out. The quality of a lesson did not depend on the theme being correct: the four cases with a mismatched theme each scored 91.7%. Story length was less consistent. Five of the 12 stories were shorter than the 200-word target, with the shortest at 170 words.

6.1.3 The effect on students (RQ3)

The results for RQ3 come from the comparison study and are reported in Section 6.3.

6.2 Product and technical results

The component passed all 15 non-functional checks. Table 30 shows the results.

Table 30:Results of the non-functional tests

Area	Check	Result
Availability	Status request responds	Passed, in 3.3 milliseconds
Availability	Chapters request responds	Passed, in 12.8 milliseconds, with four textbooks
Response time	Status within 500 milliseconds	Passed, 3.3 milliseconds
Response time	Theme classification within 800 milliseconds for 95% of requests	Passed, 32.8 milliseconds, with a mean of 28.4 over 8 requests
Configuration	Classifier in trained mode, with its two files present	Passed
Configuration	Survey loads with six questions	Passed
Configuration	Five themes published, and the returned theme is one of them	Passed
Privacy	No keys or vendor names in the classification response	Passed
Security	Settings file, index folder and test folder excluded from the repository	Passed

Table 31 shows the non-functional requirements against the evidence.

Table 31:Non-functional requirements and their results

Requirement	Result	Evidence
NFR1 Response time	Met	Status in 3.3 ms and classification in 32.8 ms, both far inside their limits
NFR2 Privacy	Met	No names stored, and no keys in responses
NFR3 Local running	Met	Both models ran through Ollama on the development computer
NFR4 Cost	Met	No per-token charge was incurred
NFR5 Security	Met	Settings file and index excluded from the repository
NFR6 Grounding	Partly met	Similarity to the textbook averaged 78.0%
NFR7 Reliability	Met	12 of 12 lessons in the required format
NFR8 Modularity	Met	The component runs as a separate module in the application

6.3 User or stakeholder validation

[This section is completed after the comparison study.]

[Number] students took part, [number] in each group. Table 6.7 shows the results.

Table 6.7: Results of the comparison study

Measure	Story lesson group	Textbook group
Number of students	[ ]	[ ]
Mean quiz score, out of [ ]	[ ]	[ ]
Mean interest rating, out of 5	[ ]	[ ]

[Two or three sentences on what the table shows: which group scored higher on the quiz, which group rated the lesson as more interesting, and how large the differences were.]

[If collected: the number of students who said their story theme matched them, out of the number who answered.]

6.4 Integration results

The component worked correctly inside the complete solution. With the full application running, it reported that it was ready and that the trained classifier was loaded. It listed all four textbooks. It generated all 12 benchmark lessons through the Ollama runtime that it shares with Voice Tutor, and in every case the retrieval step returned passages from four sources.

The tests did not record how often the shared project index was used as a backup, so that part of the integration is confirmed only by the lessons being complete.

6.5 Summary of findings

Table 32 summarises the findings against the research questions.

Table 32:Summary of findings

Question	Finding
RQ1: How accurately can the classifier predict a story theme?	54.7% in cross-validation and 8 of 12 on the benchmark. This is well above chance but limited by having only two inputs.

RQ2: How well grounded are the generated lessons?	Similarly to the textbook averaged 78.0%, and all 12 lessons were complete and correctly formatted. Llama 3.1 outperformed the baseline model on every measure.
RQ3: Do students learn better and enjoy it more with a story?	[From Section 6.3.]
System quality	All 15 non-functional checks passed.

7 DISCUSSION

7.1 Interpretation of findings

The classifier. The classifier predicted the right story theme for 76.1% of records, against 20% for a random guess. This shows that a student's interest and hoped-for job carry real information about the kind of story they prefer. The interest carried most of it.

The accuracy on the 26 real responses was lower, at 56.0%. The difference comes from the data. In the synthetic records, each interest always leads to the same theme. Real students are less predictable: those who chose Gaming and Engineer picked three different themes. This supports the idea behind the design, that a hobby is not the same as a taste in stories. It also sets a limit. A classifier with only two inputs cannot tell apart students who give the same two answers.

The most common error was between Sci-Fi & Innovation and Logic & Mystery. Both themes attract students who like coding, mathematics and gaming and who want to be engineers, so the two inputs do not separate them well.

The lessons. Llama 3.1 returned all 12 lessons in the required format, each with a three-paragraph story. Their similarity to the textbook averaged 78.0%. This shows that a model running on a local machine can produce structured, textbook-based lessons reliably when it is given retrieved passages and a fixed format. The baseline model could not: it failed the format in 4 of 12 cases and wrote three paragraphs in only 8% of them.

A wrong theme did not lower the quality score. The four lessons with a mismatched theme each scored 91.7%. The rubric measures whether the science is present and explained, not whether the story suits the student, so the effect of a wrong theme on the student is not captured by the benchmark.

Five of the 12 stories were shorter than the 200-word target. The model follows the structure more reliably than the length.

Speed. A lesson took about 44 seconds. This is acceptable for a student studying alone but too slow for use during a live lesson.

The effect on students. [Two or three sentences on the comparison study, once Section 6.3 is complete.]

7.2 Comparison with literature baselines and existing solutions

Tang and Hare used a Random Forest in a narrative game to classify students by their level of knowledge [17]. This research applied the same kind of model to a different question, the student's story preference. They report no accuracy figure, so the two cannot be compared in numbers.

Garonga and Tanduk found Naïve Bayes more accurate than Random Forest for classifying learning styles, at 75% against 59% [21]. On my data the two were close, with the Random Forest at 76.1% and Naïve Bayes at 74.1%. How the answers are encoded mattered more than the choice of classifier. When each answer was encoded as a single number, the same Random Forest reached only 54.7%.

Chen et al. found that retrieval improved answers more for a weaker open-source model than for a commercial one [8]. My results agree with this. With retrieval and a fixed format, a local open model produced complete lessons in every case.

Lim et al. personalised course material for university students and found that removing retrieval lowered accuracy and clarity [3]. This research kept retrieval in every lesson for the same reason, and applied it to school students and to stories.

Compared with the existing systems in Table 2, the component is the only one that combines a generated story, a theme predicted by a classifier, and facts taken from a school textbook. MirrorStories personalises stories but has no curriculum behind them. Oak Story and Khanmigo are tied to learning content but depend on commercial cloud models.

7.3 Research contribution

This research makes four contributions.

1. It shows that a student's story theme can be predicted from two survey answers with 76.1% accuracy, and it shows where the prediction fails and why.

2. It presents a lesson format that separates the explanation from the story, with retrieval from the official textbook, and gives benchmark evidence that the format can be produced reliably.
3. It gives evidence that a locally run open model is good enough for this task, together with a direct comparison of two such models.
4. It reports two practical findings from development. Header-only passages weakened the first index, and the way categorical answers are encoded has a large effect on a Random Forest.

7.4 Product contribution

The product is a working module inside the group's learning application. A student completes a short survey, receives a story theme, selects a textbook topic, and receives a two-part lesson with its sources. The module runs on a local machine with no per-token cost. It includes a fallback for when the classifier or the language model fails, so the student always receives a lesson. It also comes with a 12-case benchmark and a suite of 15 automated tests, which can be reused to check any future change.

7.5 Limitations and threats to validity

- **Small sample.** Only 26 real survey responses were used. They came from one source, so they may not represent Sri Lankan students in general.
- **Synthetic data.** Most of the training records are synthetic and follow simpler patterns than real students. The 76.1% figure is raised by them. The 56.0% on real responses is the better guide to performance with new students.
- **Two inputs.** The classifier uses only the interest and the hoped-for job, which limits how accurate it can be.
- **Automated measures.** The quality score and the textbook similarity are calculated by a script. No person checked each fact in each lesson against the textbook.

- **Small benchmark.** The benchmark has 12 cases, each run once. The language model does not give the same output every time, so a repeat would give slightly different scores.
- **No comparison without retrieval.** Every lesson was generated with retrieval, so the results do not show how much retrieval itself adds.
- **Expected themes.** The expected theme for each benchmark persona was set by [how they were set], which may favour the classifier.
- **One computer, one subject, one language.** The timings come from one computer, and the component covers Science in English only.
- **No bias testing.** The classifier was not tested for bias.
- **The comparison study.** [Its sample size and any other limits, once complete.]

7.6 Implications

For schools, the component shows that a personalised study aid can run without a paid AI service. A school would still need a computer able to run the model, and should treat the lessons as support for the teacher.

For developers, three design choices made a small local model usable: retrieval from a trusted source, a fixed output format, and a fallback that never invents content. The results also show that categorical survey answers should be one-hot encoded before they are given to a Random Forest.

For researchers, the gap between synthetic and real accuracy shows the need for a larger real sample. More inputs, such as the level of struggle, may help separate students who share an interest.

8 CONCLUSION AND FUTURE DIRECTIONS

8.1 Conclusion

This research set out to address two problems. A language model asked to turn science into a story tends to invent facts that are not in the textbook. Writing a personal story for every student by hand does not scale, and matching a story to a hobby alone is too crude.

To address them, I built the Narrative Learning component. It predicts a story theme for each student from two survey answers. It retrieves passages on the chosen topic from the official Grade 10 and 11 Science textbooks. It then uses a language model running on the local machine to write a lesson in two parts: a plain explanation, followed by a story matched to the student.

The results answer the research questions as follows. The classifier predicted the right theme for 76.1% of records and for 56.0% of the real survey responses, which shows that the approach works but is limited by having only two inputs. The generated lessons were complete and correctly structured in all 12 benchmark cases, with an average similarity of 78.0% to the textbook. Llama 3.1 outperformed the baseline model on every measure. The component passed all 15 of its non-functional tests. [One sentence on the result of the comparison study.]

Overall, the research shows that a personalised, textbook-based story lesson can be produced by a small system that runs on one computer with no paid AI service. The main weakness is the classifier’s accuracy on real students, which needs more real data and more inputs to improve.

8.2 Achievement of objectives

Table 33 sets out each objective from Section 1.5.2 and how far it was achieved.

Table 33:Achievement of objectives

Objective	Status	Evidence
1. Train and evaluate a classifier that predicts a story theme from interest and hoped-for job	Achieved	76.1% accuracy in cross-validation and 56.0% on real responses (Section 6.1.1)
2. Build a retrieval engine that uses the component's own index with the shared index as a backup	Achieved	Passages from four sources retrieved in all 12 benchmark cases (Section 6.4)
3. Build a generation pipeline that writes a two-part lesson	Achieved	12 of 12 lessons in the required format, all with a three-paragraph story (Section 6.1.2)
4. Integrate the component into the group's application and run it on a local model	Achieved	The component runs inside the application and passed all 15 non-functional tests (Sections 6.2 and 6.4)
5. Evaluate the classifier, the stories and the effect on students	Partly achieved	The classifier and the stories were evaluated. The stories were scored by an automated benchmark, not by a manual check of each fact. [The comparison study with students: state whether it was completed.]

8.3 Individual research and product contributions

My research contributions are:

- evidence that a student's story theme can be predicted from two survey answers, with a clear account of where the prediction fails
- a two-part lesson format that keeps the explanation separate from the story and takes its facts from the official textbook
- evidence that a locally run open model can produce these lessons reliably, with a comparison of two such models
- two practical findings: header-only passages weaken a textbook index, and the encoding of survey answers strongly affects a Random Forest

My product contributions are:

- a working Narrative Learning module, integrated into the group's learning application
- a survey, a trained classifier and its training script
- a textbook index for the component, with a fallback to the shared project index
- a lesson engine with a retry and a fallback, so that a student always receives a lesson
- a 12-case benchmark and a suite of 15 automated tests that can be reused

8.4 Future directions

REFERENCES

[1] A. M. Diana J. Arya, “The role of the scientific discovery narrative in middle school science education: An experimental study.,” *IEEE Acad. Lit.*, 2012, doi: 10.1037/A0028108.

[2] R. P. Sachin Datt, “Effect of Historical Narrative Based Approach in Designing Secondary School Science Content on Students’ Memory Recall Performance in a School in Mumbai,” *IEEE Acad. Lit.*, 2013, doi: 10.1007/978-81-322-1050-4_23.

[3] B. X. Joy Lim Jia Yin Daniel Zhang-Li, Ji-Fan Yu, Xin Cong, Ye He, Zhiyuan Liu, Hui-Qin Liu, Lei Hou, Juan-Zi Li, “Learning in Context: Personalizing Educational Content with Large Language Models to Enhance Student Learning,” *IEEE Acad. Lit.*, 2025, doi: 10.48550/arXiv.2509.15068.

[4] H. P. Waam Wanniarachchi, “Identifying the Learning Style of Students Using Machine Learning Techniques: An Approach of Felder Silverman Learning Style Model (FSLSM),” *IEEE Acad. Lit.*, 2024, doi: 10.9734/ajrcos/2024/v17i3422.

[5] W. X. Zachary Levonian Cheng-Lu Li, Wangda Zhu, A. Gade, Owen Henkel, Millie-Ellen Postle, “Retrieval-augmented Generation to Improve Math Question-Answering: Trade-offs Between Groundedness and Human Preference,” *IEEE Acad. Lit.*, 2023, doi: 10.48550/arXiv.2310.03184.

[6] Á. T. Renáta Németh Annamária Tátrai, Miklós Szabó, Péter Tibor Zaletnyik, “Exploring the use of retrieval-augmented generation models in higher education: A pilot study on artificial intelligence-based tutoring,” *IEEE Acad. Lit.*, 2025, doi: 10.1016/j.ssaho.2025.101751.

[7] G. P. Kommu Gangadhara Rao Mungara Susmitha Rani, K. Singamaneni, Katta Sugamya, “Benchmarking retrieval strategies for curriculum-aligned retrieval-augmented generation using NCERT-RAG-Eval STEM textbooks,” *IEEE Acad. Lit.*, 2026, doi: 10.1038/s41598-026-71757-y.

[8] K. K. Eason Chen Chuangji Li, Shizhuo Li, Zimo Xiao, Jionghao Lin, “Comparing RAG and GraphRAG for Page-Level Retrieval Question Answering on Math Textbook,” *IEEE Acad. Lit.*, 2025, doi: 10.48550/arXiv.2509.16780.

[9] H. S. Chelsea William Fernandes Setareh Rafatirad, “Advancing Personalized and Adaptive Learning Experience in Education with Artificial Intelligence,” *IEEE Acad. Lit.*, 2023, doi: 10.23919/EAEEIE55804.2023.10181336.

[10] J. A. L. Alan Y. Cheng Carolyn Q. Zou, Anthony Xie, Matthew Hsu, Felicia Yan, Felicity Huang, D. K. Zhang, Arjun Sharma, Rashon Poole, Daniel Wan Rosli, Andrea Cuadra, Roy D. Pea, “Oak Story: Improving Learner Outcomes with LLM-Mediated Interactive Narratives,” *IEEE Acad. Lit.*, 2025, doi: 10.1145/3746059.3747698.

[11] H.-Y. L. Pei-Hua Chen Yuen-Min Huang, Ting-Ting Wu, “Mitigating Artificial Intelligence Hallucinations in Education: A Comparative Study of Retrieval-Augmented Generation (RAG) and Large Language Models,” *IEEE Acad. Lit.*, 2025, doi: 10.1109/ICMET67594.2025.11451842.

[12] J. E. S. Elizabeth Csikar, “The Utility of Storytelling Strategies in the Biology Classroom,” *IEEE Acad. Lit.*, 2018, doi: 10.30935/CEDTECH/6210.

[13] J. V. L. Russel Fitz P. Wadasen, “Effect of Teacher-Made Narrative Fiction (TMNF) about Covalent Bonding on Students’ Achievement, Retention and Interest,” *IEEE Acad. Lit.*, 2019, doi: 10.70884/mjsir.v79i3.234.

[14] R. Maharaj-Sharma, “Using storytelling to teach a topic in physics,” *IEEE Acad. Lit.*, 2022, doi: 10.1080/20004508.2022.2092977.
[15] M. K. Samuel Tobler Tanmay Sinha, Katja Köhler, “Teaching biology with narratives: examining the impact of affective and cognitive variables on undergraduate student learning,” *IEEE Acad. Lit.*, 2025, doi: 10.1007/s11251-025-09723-w.
[16] C. A. Agyei, “Digital storytelling and students’ motivation towards learning genetics: A quasi-experimental study,” *IEEE Acad. Lit.*, 2026, doi: 10.55284/ajel.v11i2.1939.
[17] R. H. Ying Tang, “A Random Forest Model for Personalized Learning in a Narrative Game,” *IEEE Acad. Lit.*, 2020, doi: 10.18260/1-2–34041.
[18] M. P. Sushil Shrestha, “Determining Learning Style preferences of learners,” *IEEE Acad. Lit.*, 2021, doi: 10.30564/JCSR.V3I1.2761.
[19] I. W. Yunia Ikawati M. H. A. Al Rasyid, “Student Behavior Analysis to Predict Learning Styles Based Felder Silverman Model Using Ensemble Tree Method,” *IEEE Acad. Lit.*, 2021, doi: 10.24003/EMITTER.V9I1.590.
[20] M. A. F. A. Almahri David Bell, “Applications of Machine Learning in Education,” *IEEE Acad. Lit.*, 2021, doi: 10.4018/978-1-7998-5009-0.CH004.
[21] R. T. Melki Garonga, “COMPARISON OF NAIVE BAYES, DECISION TREE, AND RANDOM FOREST ALGORITHMS IN CLASSIFYING LEARNING STYLES OF UNIVERSITAS KRISTEN INDONESIA TORAJA STUDENTS,” *IEEE Acad. Lit.*, 2023, doi: 10.52436/1.jutif.2023.4.6.1020.
[22] A. E. Sarfaroz Yunusov Hamza Sidat, “MirrorStories: Reflecting Diversity through Personalized Narrative Generation with Large Language Models,” *IEEE Acad. Lit.*, 2024, doi: 10.48550/arXiv.2409.13935.
[23] M. Ruskov, “Our Coding Adventure: Using LLMs to Personalise the Narrative of a Tangible Programming Robot for Preschoolers,” *IEEE Acad. Lit.*, 2025, doi: 10.48550/arXiv.2506.20982.
[24] S. L. C. Abhishek Kulkarni S. Brown, Neha Rani, “Towards Pedagogy-Grounded Conversational AI Tutors for Interest-Based Learning,” *IEEE Acad. Lit.*, 2026, doi: 10.1145/3816046.3816215.
[25] J. L. Martin Stefanko, “A Pipeline for Source-Grounded Learning Object and Item Generation from PDF Materials Using Large Language Models,” *IEEE Acad. Lit.*, 2026, doi: 10.1109/IIT.SRC67760.2026.11629076.
[26] D. J. Arya, “Discovery stories in the science classroom,” *IEEE Acad. Lit.*, 2010, [Online]. Available: https://www.semanticscholar.org/paper/c5bc16e40c6ce81b66d79106cc59e665ebd9c59b
[27] D. A. P. S. McGee Amanda M. Durik, “The Impact of Text Genre on Science Interest in an Authentic Science Learning Environment,” *IEEE Acad. Lit.*, 2014, [Online]. Available: https://www.semanticscholar.org/paper/0068d9bda92ddc66a7bd7be921157de2f2e5ce95
[28] J. C. L. Wookhee Min Jonathan P. Rowe, Bradford W. Mott, “Personalizing Embedded Assessment Sequences in Narrative-Centered Learning Environments: A Collaborative Filtering Approach,” *IEEE Acad. Lit.*, 2013, doi: 10.1007/978-3-642-39112-5_38.

[29] D. G. Gabriel Barata S. Gama, J. Jorge, “Early Prediction of Student Profiles Based on Performance and Gaming Preferences,” *IEEE Acad. Lit.*, 2016, doi: 10.1109/TLT.2016.2541664.
[30] J. P. Rowe, L. R. Shores, B. W. Mott, and J. C. Lester, “Integrating Learning, Problem Solving, and Engagement in Narrative-Centered Learning Environments,” *Int. J. Artif. Intell. Educ.*, vol. 21, no. 1–2, pp. 115–133, Feb. 2011, doi: 10.3233/JAI-2011-019.
[31] “Khanmigo for learners: Always-available tutor, powered by AI.” Accessed: Oct. 11, 2026. [Online]. Available: https://khanmigo.ai/learners
[32] L. Breiman, “Random Forests,” *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, Oct. 2001, doi: 10.1023/A:1010933404324.
[33] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” Aug. 27, 2019, *arXiv*: arXiv:1908.10084. doi: 10.48550/arXiv.1908.10084.
[34] J. Johnson, M. Douze, and H. Jegou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, Jul. 2021, doi: 10.1109/TBDATA.2019.2921572.
[35] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 12, 2021, *arXiv*: arXiv:2005.11401. doi: 10.48550/arXiv.2005.11401.
[36] A. Grattafiori *et al.*, “The Llama 3 Herd of Models,” Nov. 23, 2024, *arXiv*: arXiv:2407.21783. doi: 10.48550/arXiv.2407.21783.

GLOSSARY

[Guidance - delete before submission: Optional. Define specialised terms that a reader may not know. Do not duplicate the list of abbreviations.]

[Term - definition.]

APPENDICES

[Guidance - delete before submission: Include supplementary evidence such as instruments, ethics documents, detailed results, additional test cases, selected technical specifications, and AI-use disclosure. Do not use appendices to avoid explaining essential material in the main thesis.]

Appendix A Research instruments or extended results

[Guidance - delete before submission: Replace the title and content as appropriate, and cite the appendix from the thesis.]

Appendix B Academic integrity and AI use disclosure

[Guidance - delete before submission: List the tools used, purpose, representative prompts or workflows where required, verification performed, and sections or artefacts affected, in accordance with module policy.]